Taskly Frontend Audit

Full Read-Only Documentation for Java/Spring Boot to Node.js Backend Migration

---

Frontend Audit and UI Change Plan

Cover Page

Project: Taskly
Audit Type: Full read-only frontend audit (React/TypeScript)
Conversion Context: Backend migration from Java/Spring Boot to Node.js — frontend must keep working
Output: `Frontend_Audit_and_UI_Change_Plan.pdf`

---

1. Folder Structure & Key Files

Category	Details
Build Tool	Vite 5 + React 18 + TypeScript 5
UI Library	`@heroui/react` v2.8.5 + Tailwind CSS 4 (HeroUI plugin)
Routing	`react-router-dom` v6 with `ProtectedRoute` pattern
State Mgmt	Zustand (persisted) + `localStorage` for auth
API Client	Direct `axios` calls — **no centralized instance**
Auth	JWT in `localStorage`, Bearer token in headers
Charts	`react-apexcharts` + custom radial bar charts
Excel	`exceljs` + `xlsx` for file generation
WebGL	`ogl` for Aurora canvas background
Notifications	Push API + Service Worker + `localStorage`
No .env files	All configs hardcoded or in `package.json`
No i18n	English-only, all texts hardcoded

Package.json Key Versions

- `axios` ^1.13.1
- `react-router-dom` ^6.23.0
- `zustand` ^5.0.12
- `@heroui/react` ^2.8.5 + all sub-packages
- `apexcharts` ^5.3.6 + `react-apexcharts`
- `framer-motion` ^11.18.2
- `ogl` ^1.0.11
- `query-string` ^9.3.1

- `date-fns` ^4.1.0
- `moment` ^2.30.1
- `exceljs` ^4.4.0 + `xlsx` ^0.18.5

Key Config Files

- `vite.config.ts` — Vite build config, `@vitejs/plugin-react`
- `tailwind.config.js` — HeroUI plugin, custom colors, dark mode `class`
- `postcss.config.js` — PostCSS setup
- `vercel.json` — Deployment config
- `src/config/site.ts` — Site navigation config (general-purpose, not Taskly-specific)
- `.gitignore`, `.npmrc`

Environment Variables

None found. All API base URLs are hardcoded:

`https://taskly.tapmind.io/t2.0/authentificationUtilization`

Every axios call uses relative paths like `/t2.0/authentificationUtilization/<endpoint>` which are resolved against this base.

---

2. Routing — Full Route Table

Public Routes

Path	Component	Notes
`/`	`IndexPage` (login)	Aurora canvas background; redirects to `/dashboard` if token exists
`/logout`	`Logout`	Removes `authUserLD` from localStorage
`/passwordHandler`	`ResetPasswordPage`	Forgot password flow

Protected Routes (behind `ProtectedLayout` = `ProtectedRoute` + `Layout` with sidebar + topbar)

Path	Component	Access Guard
`/dashboard`	`DashboardPage`	All authenticated
`/activity-log`	`ActivityLogPage`	All authenticated
`/department`	`DepartmentPage`	`adminOnly`
`/primaryDepartment`	`PrimaryDepartmentPage`	`adminOnly`
`/department-forms`	`FormPage`	All authenticated
`/tasks`	`TaskManagementPage`	`adminOnly`
`/target`	`TargetManagement`	`adminOnly`
`/efficiency-report`	`Efficiency`	All authenticated
`/access-management`	`AccessManagementPage`	`adminOnly`

/sales-forward	SalesForward	salesOnly (deptID = 21)
/Goal-management	GoalManagement	adminOrTech
/pms	PMS	All authenticated
/Entry-Approval	EntryApproval	All authenticated
/User-History	History	adminOnly
/Query-Report	QueryReport	adminOnly
/goals	Goals	All authenticated
/hr-feedback	HRFeedback	All authenticated

ProtectedRoute Guards

Role Check	Condition
adminOnly	user.userRole === 1
salesOnly	user.departmentID === 21
HrOnly	user.departmentID === 22
TechOnly	user.departmentID === 24
adminOrTech	user.userRole === 1 OR user.departmentID === 24

Sidebar Menu Filtering

- Defined in `src/components/sidebar.tsx` (427 lines)
- Filtered by: `user.userRole`, `user.departmentID`, `rolePermissions`
- Menu items with `adminOnly`, `salesOnly`, `departments` props are filtered accordingly
- Role-per-module check: `rolePermissions?.[module]?.includes("read")`

Topbar User Display

- Shows user name from `authUserLD`
- Shows department name (parsed and title-cased)
- Avatar with dropdown: Profile, Logout
- Token expiry timer: auto-logout if JWT `exp` elapsed

---

3. State Management & API Layer

Zustand Store (`src/store/commonStore.tsx`)

- Single store: `useDashboardStore`
- Persisted to localStorage key `"dashboard-storage"`
- Fields: `dateModification` (boolean), `allUser` (User[])
- Used by `DashboardPage` only

Auth State (localStorage)

Key	Contents
`authUserLD`	User profile: `userID`, `userName`, `userEmail`, `departmentName`, `departmentID`, `role`, `roleID`, `token`, `isActive`, `userCreateTime`
`roles`	Role privileges: `JSON.parse(rolePrivileges)` — stores per-module permissions
`pushEnabled`	Flag: `"true"` / `"false"`

API Layer — Direct axios calls (NO centralized instance)

- **Base URL:** `https://taskly.tapmind.io/t2.0/authentificationUtilization`
- **Pattern:** `axios.post("/t2.0/authentificationUtilization/<endpoint>", body, { headers })`
- **Auth header:** `Authorization: Bearer <token>` when token exists
- **Content-Type:** `application/json` on all POST calls

Common API Pattern

```
axios.post("/t2.0/authentificationUtilization/<endpoint>", body, {
  headers: { "Content-Type": "application/json" }
});
// With auth:
axios.post("/t2.0/authentificationUtilization/<endpoint>", body, {
  headers: {
    "Content-Type": "application/json",
    Authorization: `Bearer ${token}`
  }
});
```

Token Expiry Handling

- Topbar (`src/components/topbar.tsx` lines 83-104): On mount, decodes JWT payload, checks `exp * 1000 - Date.now()`
- If expired → `handleLogout()` → `localStorage.removeItem("authUserLD")` + `window.location.href = "/"`
- Timer setTimeout auto-logs out

401/403 Handling

- **Not explicitly handled** in API layer — relies on route guards (`ProtectedRoute`)
- If token expires, topbar auto-logout redirects to `/`
- No interceptor to catch 401 and refresh token / show error

Token Storage

- JWT stored in `localStorage.getItem("authUserLD")` as full JSON object
- Token accessible via `JSON.parse(authUserLD).token`
- No secure HTTP-only cookie usage

---

4. Global Providers, Layouts, Theme, i18n

Providers (`src/provider.tsx`)

```
<HeroUIProvider navigate={navigate} useHref={useHref}>
  <ToastProvider />
  {children}
</HeroUIProvider>
```

- `HeroUIProvider` from `@heroui/system` — provides theming and navigation context
- `ToastProvider` from `@heroui/toast` — toast notifications

Layout (`src/components/layout.tsx`)

- Sidebar (collapsible on mobile)
- Topbar (fixed, with user profile + notifications)
- Main content area with padding `p-6`
- Dark mode: `dark:bg-gray-900` on body

Theme (`tailwind.config.js`)

- HeroUI plugin configured
- Custom colors: primary `#FE5F00`, secondary, danger, success, neutral
- Dark mode: `class` strategy
- Font: `Poppins`
- Keyframes: "shine" animation for Aurora background

i18n

- **None detected** — all text hardcoded in English
- No `react-i18next` or similar
- All UI strings are literal values in component files

---

5. Pages/Modules Summary (23 pages)

#	Page	Route	Purpose
1	Login	`/`	Authentication form
2	Dashboard	`/dashboard`	Main dashboard with radial charts, stats, department lists
3	Activity Log	`/activity-log`	Daily activity logging via `DailyActivityLog` component
4	Department	`/department`	Department management (add/edit/delete/sub-departments)
5	Efficiency Report	`/efficiency-report`	Efficiency summary and table
6	Entry Approval	`/Entry-Approval`	Task/goal/target approval workflow
7	Forgot Password	`/passwordHandler`	Password reset flow

8	Forms	`/department-forms`	Department form submission
9	Goal Management	`/Goal-management`	Department goal CRUD
10	Goals Report	`/goals`	Goal progress report across users/ departments
11	History	`/User-History`	User performance history
12	Access Management	`/access-management`	User role/permission management
13	Sales Forward	`/sales-forward`	Sales call scheduling/rescheduling (deptID = 21)
14	PMS	`/pms`	PMS dashboard, goal setting, reviews
15	Primary Department	`/primaryDepartment`	Link primary department to sub-department
16	Query Report	`/Query-Report`	Filterable performance query report
17	Reports	`/reports`	Daily task reports
18	Target Management	`/tasks`	Target/Task CRUD per department
19	Daily Logs	`/daily-log`	Daily activity logs with filtering
20	HR Feedback	`/hr-feedback`	HR modules (delegates to external HR repo)
21	Department Forms (duplicate)	—	See Forms page
22	Efficiency (duplicate)	—	See Efficiency report
23	Index/Login	`/`	Login page

> **Note:** Pages 21-22 appear to be duplicates of pages 5 and 8 based on file names but have different implementations.

---

6. Hardcoded API Base URL

`https://taskly.tapmind.io/t2.0/authentificationUtilization`

All 60+ API endpoints use this base with relative paths. The entire frontend will need to adapt if the Node.js backend changes this base URL or response shapes.

DashboardPage (src/pages/dashboard.tsx)

Route: `/dashboard`
Access: All authenticated users
File: `src/pages/dashboard.tsx` (705+ lines)

Purpose & User Flow

The dashboard is the main landing page after login. It provides:

- Radial bar charts showing target %, allocated hrs, work hrs, and goal efficiency
- Summary stats: "Target Achieved (Overall)", "Allocated hours per day (Average)", "Working hours per day (Average)", "Overall Goal"
- Department list with filterable user lists
- Report generation (calculateAll, download Excel)
- Missing data log generation
- Date range picker for filtering all data

User flow:

1. Login → dashboard loads with default 7-day range
2. User can change date range via CustomDateRangePicker
3. User can filter by department or view all users
4. User can run "Calculate Report" or download Excel
5. User can view department summaries and individual user efficiency

---

API Calls

#	Method	Endpoint	Request Payload	Response Fiel
1	POST	/t2.0/authentificationUtilization/DashBoardAll	payload object containing: startDate, endDate, goalStartDate, goalEndDate, departmentType, roleID, userID, isHeExecutiveLeader, isHeFunctionalLeader	elSummary, flS elData (if role = departmentSum userSummary, companySummar (efficiency levels, percentages), elUsersSummar elUsersSummar flUsersSummar flUsersSummar allUser
2	POST	/t2.0/authentificationUtilization/Department/list?page=0&size=0	roleID: user.userRole, userID: ...	Department list w departmentID, departmentNam departmentDes newStatus
3	POST	/t2.0/authentificationUtilization/users/read?page=0&size=0	filter, searchText, departmentID, roleID, userID, token	User list: userID userName, role
4	GET	/t2.0/authentificationUtilization/calculateAll	None	Success toast
5	GET	/t2.0/authentificationUtilization/MissingData?startDate=&endDate=	startDate, endDate	Log data array
6	POST	/t2.0/authentificationUtilization/DashBoardAll/download/excel	Same payload as #1	Excel blob

Forms

Form	Fields	Validation	Defaults	Submit Behavior
Date Range Picker	startDate, endDate	None (free text via calendar)	Last 7 days from today	Updates `dateRange` state, triggers data refetch
CustomDateRangePicker	Same as above	None	—	—

Tables/Lists

Table	Columns	Sorting	Filtering	Pagination	Export/ Download
Department list (internal)	departmentID, departmentName, description, status	—	By searchTerm (not exposed UI)	—	—
User summary table	Target Achieved %, Utilization %, Allocated Hrs, Total Work	—	By department filter, user role	—	Excel download (via calculateAll)
Company summary radial chart	5 radial metrics (Target, Allocated, Work, Goal, Efficiency)	—	Date range filter	—	Excel download

State

Type	Source	Description
Local state	`useState` in DashboardPage	`dateRange`, `goalDateRange`, `refresh`, `selectedDepartment`, `elFilter`, `flFilter`, `departmentType`, `pageNumber`, `pageSize`, `searchValue`
Global store	`useDashboardStore`	`dateModification`, `allUser` (persisted)
Derived	`useMemo/useCallback`	`stats` computed from `companySummary.values`, `ELSummaryFiltered`, `FLSummaryFiltered`, `departmentSummaryFiltered`
Effects	`useEffect` in `useDashboardData` hook	Fetches data when `payload` changes; aborts with `AbortController`

Effects & Dependencies

- `useDashboardData` hook (lines 443-616): Dependencies: `[payload, refresh]` — fetches `/DashBoardAll`; aborts on unmount
- Department fetch (line 921): No dependencies — runs once on mount
- Users fetch (line 952): Dependency: `[selectedDepartment]` — fetches users for selected dept
- Report fetch (line 993): No dependencies — runs on button click
- Download (line 1026): Dependency: `[payload]` — downloads Excel

UI Components Used (tree, defined in)

```
DashboardPage
├── ColumnChart (inline, lines 67-231) — radial bar chart variant
├── StatRow (inline, lines 362-384) — stat card component
├── EfficiencyRules (inline, lines 387-429) — rules explanation panel
├── RadialBarSkeleton (inline, lines 435-439) — loader
├── RadialSummaryCard (memo, lines 659-720) — radial chart card
├── MonthYearPicker (memo, lines 722-841) — month/year selector
├── CustomDateRangePicker (component, ../components/CustomDateRangePick
├── EfficiencyTable (component, ../components/table2) — efficiency tabl
├── ElDashboard (component, ../components/ElDashboard) — executive dash
├── SwitchUserFilter (component, ../components/SwitchUserFilter) — user
├── SwitchedUserBanner (component, ../components/SwitchedUserBanner) —
├── addToast (from @heroui/react) — toast notifications
├── Dropdown, DropdownMenu, DropdownItem (from @heroui/react) — date ra
├── Tabs, Tab (from @heroui/react) — executive leader tabs
├── Select, SelectItem (from @heroui/react) — department/user selectors
├── Button (from @heroui/react) — action buttons
├── motion, AnimatePresence (framer-motion) — chart animations
├── EmptyContainer (component) — empty state
```

Known Defects, Dead Code, TODOs

1. **Line 92-93**: `STAT_DEFS` has commented-out entries — some labels/keys are commented
2. **Lines 212-216**: `ColumnChart` has commented-out grid stats section
3. **Lines 234-257**: `getEfficiencyLevel` function is completely commented out
4. **Line 519**: `if (user.userRole === 3)` branch in `useDashboardData` references `data.elGoalSummary` which may be undefined
5. **Line 906-918**: `stats` memo depends on `companySummary` but uses `def.key >= 0` check — `def.key === "subtitle"` branch may fail if `companySummary.subtitle` is undefined
6. **Multiple console.log statements** in the code (e.g., line 199 in subdepartment.tsx, line 383 in goals.tsx, etc.)
7. **Line 92-93 in dashboard**: `STAT_DEFS` has a comment `// const getEfficiencyLevel =` — dead code
8. **Radial chart trend is hardcoded** `" +12.4%"` (line 92) — not dynamic
9. **No error boundary** — fetch errors caught and logged only

---

Hardcoded Values

- `7` — default date range days (line 856, 857)
- `" +12.4%"` — hardcoded trend percentage (line 92)
- `100`, `80`, `50` — efficiency thresholds in `EfficiencyRules` component
- `8` — standard workday hours mentioned in EfficiencyRules
- `0.07` — animation delay multiplier in ColumnChart
- `120` — spring stiffness in ColumnChart animation
- `18` — spring damping in ColumnChart animation

Files That Could Change Due to Backend Migration

- **API endpoints**: All `/t2.0/authentificationUtilization/...` calls — if Node.js backend changes field names or response shapes, the frontend mappings will break
- **Dashboard payload** (useDashboardData): Object with `departmentType`, `roleID`, `userID`, `isHeExecutiveLeader`, `isHeFunctionalLeader` — if backend changes these field names or adds/removes fields, the payload construction and response mapping will need updates
- **Department list response**: Maps `departmentID`, `departmentName`, `departmentDescription`, `newStatus` — if Node.js changes these, the `dept` mapping in dashboard (line 936-941) and other pages will break
- **User summary response**: Maps `UserName`, `EfficiencyLevel`, `AvgTargetPercent`, `AvgUtilizationPercent`, `AvgAllocatedHrs`, `AvgTotalWork` — critical for chart data
- **Company summary**: `EfficiencyLevel`, `AvgTargetPercent`, `AvgUtilizationPercent`, `AvgAllocatedHrs`, `AvgTotalWork`, `AvgGoalEfficiency` — used in radial charts and stats

---

ActivityLogPage (src/pages/activity-log.tsx)

Route: `/activity-log`
Access: All authenticated users
File: `src/pages/activity-log.tsx` (11 lines — very thin wrapper)

Purpose & User Flow

This page is a simple wrapper that renders the `DailyActivityLog` component. It appears to be a minimal page with minimal logic.

API Calls

- **None directly** — all API calls are handled internally by the `DailyActivityLog`

component (which was not fully read but is imported from `src/components`)

Components Used

- `DailyActivityLog` — imported from `src/components` (not individually read in detail)

Known Defects / Observations

1. **Page is essentially a pass-through** — no API calls, no state management of its own
2. May need `DailyActivityLog` component audit if deeper investigation required

---

Backend Migration Impact

- **Low risk** — this page doesn't make direct API calls; any changes would be in the `DailyActivityLog` component
- If `DailyActivityLog` uses hardcoded API paths, those would need updating

---

DepartmentPage (src/pages/subdepartment.tsx)

Route: `/department`
Access: `adminOnly` guard (requires `user.userRole === 1`)
File: `src/pages/subdepartment.tsx` (902 lines)

Purpose & User Flow

Department management page — allows admins to:

- View department list with search/filter
- Add new departments
- Edit existing departments
- Delete departments
- Toggle department status (activate/deactivate)
- Link primary departments

User flow:

1. Admin views department list
2. Clicks "Add Sub Department" → opens modal
3. Fills department name/description, selects executive/functional leaders from user list
4. Submits → creates department, refreshes list
5. Clicks edit on a department → opens edit modal

6. Updates fields → saves changes
7. Clicks delete → shows confirmation modal
8. Clicks toggle (activate/deactivate) → updates status

API Calls

#	Method	Endpoint	Request Payload	Response Fie
1	POST	/t2.0/authentificationUtilization/PrimaryDepartment/list	{}	primaryDepar primaryDepar
2	POST	/t2.0/authentificationUtilization/users/read?page=0&size=0	filter, searchText, departmentID: null, token	User list: userI userName
3	POST	/t2.0/authentificationUtilization/Department/list?page=0&size=0	searchText, filter, startDate, endDate, roleID: user.userRole, userID: ...	Department list: departmentID departmentNa departmentDe newStatus, departmentSt executiveLea functionalLe
4	POST	/t2.0/authentificationUtilization/Department/create	departmentName, departmentDescription, executiveLeaderID	Success toast
5	POST	/t2.0/authentificationUtilization/Department/update/${id}	departmentName, departmentDescription, functionalLeaderID, executiveLeaderID	Success toast
6	POST	/t2.0/authentificationUtilization/Department/archive	departmentIDList: [id]	Success toast
7	POST	/t2.0/authentificationUtilization/Department/status	departmentIDList: [id], departmentStatus: newStatus	Success toast

---

Forms

Form	Fields	Validation	Defaults	Submit Behavior
Add/Edit Department Modal	departmentName (required), departmentDescription, executiveLeader (from users list), functionalLeader (from users list)	Name required	Empty strings; leaders selected from user dropdown	On submit: creates or updates department, refreshes list

Link Primary Department Modal	Primary Department (from dropdown)	—	—	On submit: links primary dept to current dept

---

Tables/Lists

Table	Columns	Sorting	Filtering	Pagination	Export/ Download
Department list	Name, ID, description, status, actions (edit/ delete/toggle/link)	—	Search term (input field)	—	—
User list (for leaders)	User name, ID	—	Search term, department filter	—	—

Department list columns (rendered UI):

- Department name (truncated to 2 lines)
- Department ID
- Status pill: green = "Active", red = "Inactive"
- Dropdown actions: Edit, Delete, Toggle (Activate/Deactivate), Link Primary Department

---

State

Type	Source	Description
Local state	Multiple `useState` calls	`sidebarOpen`, `searchTerm`, `filterStatus`, `department`, `departmentDescription`, `users` (array), `executiveLeader`, `functionalLeader`, `departmentList`, `primaryDepartments`, `linkModal`, `selectedDepartmentId`, `edit`, `term`, `toggleConfirm`, `deleteConfirm`, `tasks` (commented out)
Derived	`useMemo`	`activePage` (from currentPage)
Effects	Multiple `useEffect` calls	Fetch primary depts on mount; fetch users on mount; fetch departments whenever `edit` changes; handle delete/toggle/logic

---

Effects & Dependencies

- Line 112-130: `fetchPrimaryDepartments` — runs once on mount
- Line 168-210: `fetchUsers` — runs once on mount
- Line 213-257: `fetchDepartments` — dependency: `[edit]` — runs whenever edit state changes
- Line 326-397: `handleAddDepartment` — on submit, creates dept + refreshes list
- Line 399-441: `handleEditDepartment` — on submit, updates dept
- Line 443-477: `handleDeleteDepartment` — on confirm, archives dept
- Line 479-517: `handleToggleDepartment` — on confirm, updates status

UI Components Used (tree)

```
DepartmentPage
├── ConfirmDeleteModal (inline, lines 22-60) — delete confirmation
├── Modal (from @heroui/react) — multiple modals: add, edit, delete, to
├── ModalContent, ModalHeader, ModalBody, ModalFooter (from @heroui/rea
├── useDisclosure (from @heroui/react) — for link modal
├── Input (from @heroui/input) — department name, description
├── Select, SelectItem (from @heroui/select) — executive leader, functi
├── Autocomplete, AutocompleteItem (from @heroui/react) — user selector
├── Button (from @heroui/button) — add, edit, delete, toggle actions
├── Dropdown, DropdownMenu, DropdownItem (from @heroui/react) — departm
├── addToast (from @heroui/react) — success/error notifications
├── Sidebar (component, ../components/sidebar) — role-filtered sidebar
├── Topbar (component, ../components/topbar) — top navigation
├── EmptyContainer (component) — when no departments
├── Chip (from @heroui/react) — status pills
```

Known Defects, Dead Code, TODOs

1. **Lines 258-285**: `fetchDepartmentTask` and related `fetchTasks` are **commented out** — dead code
2. **Lines 288-324**: Another `fetchTasks` block is **commented out** — dead code
3. **Line 74**: `tasks` state initialized as empty array `[]` with no type annotation
4. **Line 77-79**: `users` state initialized as `User[]` with `[...]` — TypeScript may infer incorrectly
5. **Line 97**: `tasks` state type `Task[]` initialized as `[]` — same issue
6. **Line 174**: `filterStatus` defaults to `user.departmentID` as string, but department list may be empty initially
7. **Line 223**: `bodyData.userID` uses `user.isHeExecutiveLeader ? Number(user.userID) : Number(filterStatus)` — conditional logic
8. **Line 83-84**: `toggleConfirm` and `deleteConfirm` initial states have `id?: number` and `name?: string` — may have type issues
9. **Line 277-281**: `handleLinkPrimaryDepartment` uses `JSON.parse(authUserLD).token` without null check (could throw if authUserLD is null)
10. **Line 199**: `console.log(formatted, "hello")` — debug line left in
11. **Line 201**: `console.log("Users formatted:", formatted)` — debug line left in
12. **Line 250**: `console.log("departmentlist", formatted)` — debug line left in
13. **The commented-out task fetch effects** (lines 258-324) suggest incomplete feature or WIP

Backend Migration Impact

High risk — this page makes direct API calls to:

- `/PrimaryDepartment/list` — response mapped to `primaryDepartmentId`,

`primaryDepartmentName`
- `/users/read` — response mapped to user list with `userID`, `userName`
- `/Department/list` — response mapped to department objects with `departmentID`, `departmentName`, `departmentDescription`, `newStatus`, `departmentStatus`, `executiveLeaderID`, `functionalLeaderID`
- `/Department/create`, `/Department/update`, `/Department/archive`, `/Department/status` — CRUD operations

If Node.js backend changes:

- Department response field names → the `dept` mapping (lines 238-246, 376-384) and all form mappings will break
- User response field names → the user list formatting (lines 191-197, 261-291) will break
- Status field names (`newStatus` vs `departmentStatus`) → toggle logic will break
- Primary department field names → link modal will break

Additional concerns:

- `authUserLD` parsing without null check in multiple places (lines 66, 137, 138, 336, 337, 411, 412) — if backend changes token format, could crash
- Role-based logic: `adminOnly` guard on route + `authUserLD?.userRole === 1` checks — if role naming changes, guards need updating
- `departmentStatus` field used in UI status pills — if backend changes this to boolean/inverted, UI display will be wrong

---

Efficiency Page (src/pages/efficiency.tsx)

Route: `/efficiency-report`
Access: All authenticated users
File: `src/pages/efficiency.tsx` (705 lines)

Purpose & User Flow

Performance insights page showing efficiency summaries across departments and users.
Features:

- Department list selector
- User filter (by user, efficiency level, department)
- Date range picker (7/30 days or custom)
- Summary card showing avg working hours, allocated hrs, target %, utilization %
- Efficiency table with per-user or per-EL/FL data
- Download Excel functionality
- Tabbed view for EL (Executive Leader) and FL (Functional Leader) summaries

User flow:

1. User selects date range
2. User can filter by department and/or specific user

3. Summary card shows aggregated metrics
4. Table shows detailed data below
5. User can download Excel report
6. EL/FL users can switch tabs to view their subordinates' data

---

API Calls

#	Method	Endpoint	Request Payload	Response Fields U
1	POST	/t2.0/authentificationUtilization/Department/list?page=0&size=0	searchText, filter, startDate, endDate, roleID: user.userRole, userID: ...	Department list: departmentID, departmentName newStatus
2	POST	/t2.0/authentificationUtilization/users/read?page=0&size=0	filter, searchText, departmentID, token	User list: userID, userName, roleI reportingManag elManagerId, functionalLead
3	POST	/t2.0/authentificationUtilization/list-efficiency-summary	departmentID, startDate, endDate, userID, groupBy: "day", isHeExecutiveLeader, efficiencyLevel	summary (AvgTargetPercent, AvgUtilizationPerc AvgAllocatedHrs, AvgTotalWork, EfficiencyLevel), da array
4	POST	/t2.0/authentificationUtilization/download-efficiency-summary	Same as #3	Excel blob

---

Forms

Form	Fields	Validation	Defaults	Submit Behavior
Date Range Dropdown	startDate, endDate	None	Last 7 days from today	Updates dateRange state
CustomDateRangePicker	Same	None	—	—
Filter Dropdown (sub-component)	department, user, efficiency level	—	—	Updates filter state, triggers refetch

---

Tables/Lists

Table	Columns	Sorting	Filtering	Pagination	Export/ Download

Efficiency table (ReusableTable)	User ID, User Name, Work Date, Target Achieved %, Allocated Time (hrs), Total Time Taken (hrs)	✓ (via ReusableTable)	✓ (via FilterDropdown)	✓ (rows per page)	✓ (Download button, Excel)
Department summary cards	Avg Target %, Avg Allocated Hrs, Avg Utilization %, Avg Target %	—	—	—	—

Efficiency table columns (from table2.tsx):

- UserID, UserName, WorkDate, TargetAchievedPercent, AllocatedTimeHrs, TotalTimeOfWork
- Color-coded efficiency levels: High (success), Medium (warning), Low (danger)

State

Type	Source	Description
Local state	Multiple `useState`	`sidebarOpen`, `dateSelect`, `departmentList`, `users`, `pageNumber`, `totalRecords`, `pageSize`, `searchValue`, `average`, `dashboard`, `loading`, `dateRange`, `userSearch`, `filteredUsers`, `efficiencyLevels`, `efficiencyFilter`, `selectedUser`
Derived	`useMemo`	`filteredUsers` computed from `users` + `userSearch`
Effects	Multiple `useEffect`	Fetch departments on mount; fetch users when `filterStatus` changes; fetch dashboard data when `filterStatus`, `dateRange`, `selectedUser`, `efficiencyFilter` change

---

Effects & Dependencies

- Line 190-228: `fetchDepartments` — runs once on mount
- Line 270-318: `fetchUsers` — dependency: `[filterStatus]` — runs when filter changes; has commented-out fetchLogs
- Line 320-393: `fetchUsers` (second one) — dependency: `[filterStatus]` — fetches users with role-based filtering (EL/FL/Reporting Manager)
- Line 395-439: `fetchDashboard` — dependency: `[filterStatus, dateRange, selectedUser, efficiencyFilter]` — fetches efficiency summary
- Line 442-491: `handleDownload` — dependency: `[filterStatus, dateRange, selectedUser, efficiencyFilter, user]` — downloads Excel

UI Components Used (tree)

```
EfficiencyPage
├── Dropdown (from @heroui/react) — date range and status dropdowns
├── DropdownTrigger, DropdownMenu, DropdownItem (from @heroui/react)
├── Button (from @heroui/button) — various actions
├── CustomDateRangePicker (component, ../components/CustomDateRangePick
├── EfficiencyTable (component, ../components/table2) — main data table
├── FilterDropdown (component, ../components/FilterDropdown) — filter p
├── ReusableTable (component, ../components/table) — generic table (not
├── addToast (from @heroui/react) — notifications
├── Tabs, Tab (from @heroui/react) — EL/FL tab switching (partially com
├── Select, SelectItem (from @heroui/select) — department, user, effici
├── Card, CardBody (from @heroui/card) — summary cards
├── Picker (from @heroui/react) — date select component (imported as Da
├── useMemo, useCallback (from react) — state management
├── format from date-fns — date formatting
```

Known Defects, Dead Code, TODOs

1. **Lines 105-114**: `reportColumns` and `reportActions` are **commented out** — dead code
2. **Lines 137-188**: `renderReportCell` function is **commented out** — dead code
3. **Lines 230-318**: `fetchLogs useEffect` is **commented out** — dead code
4. **Lines 320-393**: Second `fetchUsers` effect has `console.log` statements (lines 383-385)
5. **Lines 492-501**: `efficiencyLevel` calculation is **commented out** at bottom of file
6. **Line 71**: `isELORUSER = [2, 3, 4, 5, 6, 7, 23].includes(user?.userRole)` — hardcoded role numbers
7. **Line 73**: `filterStatus` defaults to `user.departments ? 0 : ""` — conditional based on user departments array
8. **Lines 559-562**: Filter dropdown is **conditionally rendered**: `user.userRole !== 2 && user.userRole !== 5 && user.userRole !== 6 && user.userRole !== 23` — excludes certain roles from filtering
9. **Line 562-571**: FilterDropdown receives `filteredUsers.map(u => ({ key: u.userID, name: u.userName }))` — but `filteredUsers` may not include all users
10. **Line 278**: `departmentId: filterStatus ? Number(filterStatus) : 28` — hardcoded department ID 28 as fallback
11. **Multiple `console.log` statements** (lines 383-385 in goals.tsx-style patterns)
12. **Efficiency thresholds hardcoded** in summary cards (lines 650-665): `High` at ≥100% target, `At Par` at ≥80%, `Low` otherwise — these are frontend calculations that may disagree with backend

---

Backend Migration Impact

High risk — this page makes direct API calls to:

- `/Department/list` — response mapped to department list with `departmentID`, `departmentName`, `newStatus`
- `/users/read` — response mapped to users with `userID`, `userName`, `roleID`, `reportingManagerID`, `elManagerId`, `functionalLeaderID`
- `/list-efficiency-summary` — **CRITICAL**: Response shape includes `summary` object with `AvgTargetPercent`, `AvgUtilizationPercent`, `AvgAllocatedHrs`, `AvgTotalWork`, `EfficiencyLevel` and `data` array
- `/download-efficiency-summary` — Excel download endpoint

If Node.js backend changes:

- **Efficiency summary response**: The most critical — `summary` object fields (`AvgTargetPercent`, `AvgUtilizationPercent`, `AvgAllocatedHrs`, `AvgTotalWork`, `EfficiencyLevel`) and `data` array structure must match exactly. Frontend does frontend calculations for efficiency levels but depends on these fields being present.
- **User response**: `reportingManagerID`, `elManagerId`, `functionalLeaderID` field names — if changed, user filtering logic breaks
- **Department response**: `newStatus` field name — if changed, department list display breaks
- **Date formats**: Frontend uses `format(new Date(...), "dd MMM yyyy")` — if backend sends different date formats, formatting may break
- **Efficiency calculation disagreement**: Frontend has its own efficiency level logic (lines 492-501, commented out, but the logic exists):
  - `High`: AvgTargetPercent >= 100 AND AvgUtilizationPercent >= 80 AND AvgTotalWork <= AvgAllocatedHrs
  - `At Par`: AvgTargetPercent >= 80 AND AvgUtilizationPercent >= 50 AND AvgTotalWork <= AvgAllocatedHrs
  - `Low`: all other conditions
  - If backend changes these thresholds or calculations, the frontend will display incorrect efficiency levels

Additional concerns:

- `isELORUSER` hardcoded role list `[2, 3, 4, 5, 6, 7, 23]` — if backend changes role IDs, this will misidentify users
- `filterStatus` default of `0` vs `""` — inconsistent behavior based on whether user has departments
- Hardcoded department ID `28` as fallback (line 278) — if dept 28 doesn't exist, may show wrong data
- Download button disabled when `user.departmentID` exists — may need re-evaluation if backend changes department handling

---

EntryApproval Page (src/pages/EntryApproval.tsx)

Route: `/Entry-Approval`

Access: All authenticated users
File: `src/pages/EntryApproval.tsx` (1200 lines)

Purpose & User Flow

Task/goal/target approval management. Users can:

- View combined approval list of tasks, targets, and goals pending approval
- Filter by department, user, entry type (Task/Target/Goal)
- Select multiple rows for batch actions
- Approve or reject selected entries
- View details in a drawer/update form

User flow:

1. Page loads with combined approval list
2. User can filter by department, user, entry type, date range
3. User selects one or more rows (table selection)
4. User clicks "Approve" or "Reject"
5. If rejecting: enters comment, submits rejection
6. If approving: submits approval, list refreshes
7. Selected rows can be opened in a drawer for update

API Calls

#	Method	Endpoint	Request Payload	Response Fields Used

1	POST	/t2.0/authentificationUtilization/combined/list?page=0&size=0	userID, departmentID, startDate, endDate, entryType	data array with entries array — each entry has id, type, taskName, goalName, targetName, departmentName, departmentID, workDate, userName, remarks, status, approvedByEL, commentForRejection, leaveType, dailyTaskLogID, targetID, goalID, userGoalDataID, allocatedTime, timeTakenToWork, setUp, units, achievedUnits, totalUnitValues, isForTarget, isHalfDayLeave	C
2	POST	/t2.0/authentificationUtilization/combined/approve	departmentID, taskEntryIDs (array), goalEntryIDs (array)	Success toast	V
3	POST	/t2.0/authentificationUtilization/combined/reject	id, type, comment	Success toast	V
4	POST	/t2.0/authentificationUtilization/Department/list?page=0&size=0	searchText, filter, startDate, endDate, roleID: user.userRole, userID: ...	Department list: departmentID, departmentName, description, status	C
5	POST	/t2.0/authentificationUtilization/users/read?page=0&size=0	filter, searchText, departmentID, token	User list: userID, userName	C

Forms

Form	Fields	Validation	Defaults	Submit Behavior

Reject Entry Modal	rejection comment (Textarea, minRows = 4)	Comment required for cancellation	Empty string	On submit: sends reject API call, refreshes list
Update Form (Drawer)	Passes `task` object to `UpdateForm` component	—	—	Submits updated task data
Filter Dropdown	department, user, entry type, date range	—	—	Updates filter state

---

Tables/Lists

Table	Columns	Sorting	Filtering	Pagination	Export/ Download
Combined approval table	Date, User, Entry Type, TypeID, Task/ Goal/Name, Allocated, Worked, Units, Status, Comment, Remarks, Actions	✓ (via ReusableTable)	✓ (department, user, entry type filters)	✓ (rows per page)	—

Approval columns (from `approvalColumns` array, lines 210-402):

- Date (workDate)
- User (userName)
- Entry Type (Goal/Target/Task — rendered based on goalID/targetID/taskID)
- TypeID (goalID || targetID || taskID || "-")
- Name (goalName || targetName || taskName || "-")
- Allocated (goalID → fixedGoal; else allocatedTime)
- Worked (goalID → achievedGoal; else timeTakenToWork)
- Units (goalID → achievedGoal; else achievedUnits)
- Status (chip: Pending/Approved/Rejected with color coding)
- Comment (item.commentForRejection)
- Remarks (item.remarks)
- Actions (— — edit button commented out)

Table actions (lines 115-153):

- Edit: opens selected item in drawer (but edit button code is commented out in the table rendering)
- Delete: marks row for deletion

State

Type	Source	Description

Local state	Multiple `useState`	`sidebarOpen`, `isUpdateOpen`, `selectedUpdateItem`, `selectedDepartmentId`, `rejectModalOpen`, `rejectComment`, `departmentList`, `users`, `isFilterOpen`, `tableData`, `pageNumber`, `totalRecords`, `pageSize`, `searchValue`, `selectedRows`, `loading`, `dateRange`, `entryTypeFilter`, `userSearch`
Derived	`useMemo`	`filteredUsers` from `users` + `userSearch`
Effects	Multiple `useEffect`	Fetch departments on mount; fetch users when `filterStatus` changes; `getCombinedApprovalList` triggered on `[filterStatus, selectedUser, pageNumber, pageSize, dateRange, entryTypeFilter]`

Effects & Dependencies

- Line 404-450: `fetchDepartments` — runs once on mount
- Line 452-470: `fetchTasks` (for update) — dependency: `[selectedDepartmentId]` — fetches tasks by department
- Line 474-513: `fetchUsers` — dependency: `[filterStatus]` — fetches users for department filter
- Line 756-780: `getCombinedApprovalList` — dependency: `[filterStatus, selectedUser, pageNumber, pageSize, dateRange, entryTypeFilter]` — the main data fetch; fires on every dependency change
- Line 848-904: `handleToggleUserStatus` — on confirm, updates user status via API

UI Components Used (tree)

```
EntryApproval
├── ReusableTable (component, ../components/table) — main approval tabl
├── Modal (from @heroui/react) — multiple: reject confirmation, update
├── ModalContent, ModalHeader, ModalBody, ModalFooter (from @heroui/rea
├── useDisclosure (from @heroui/react) — for modals
├── Textarea (from @heroui/input) — rejection comment
├── Button (from @heroui/button) — approve, reject, cancel, add, filter
├── Dropdown, DropdownMenu, DropdownTrigger, DropdownItem (from @heroui
├── FunnelIcon (from @heroicons/react) — filter icon
├── Chip (from @heroui/react) — status chips in columns
├── Autocomplete (from @heroui/react) — department, user, entry type fi
├── AutocompleteItem (from @heroui/react) — autocomplete options
├── Drawer, DrawerContent, DrawerHeader, DrawerBody, DrawerFooter (from
├── addToast (from @heroui/react) — notifications
├── UpdateForm (component, ../components/updateForm) — update form insi
├── Select, SelectItem (from @heroui/select) — department, role, entry
├── TableColumn, TableAction (from ../components/table) — table config
├── Format from date-fns — date formatting
├── FunnelIcon, ArrowDownTrayIcon (from @heroicons/react/24/outline) —
```

Known Defects, Dead Code, TODOs

1. **Lines 367-396**: `approvalColumns` has a **commented-out `renderCell` for actions** — the entire actions column rendering is commented out (lines 367-400)
2. **Line 117-118**: The `actions` column in `approvalColumns` has the `renderCell` completely commented out
3. **Lines 397-401**: The `userActions` table action definitions exist but the "Update" button in the footer only works for single-row selection and has complex logic
4. **Line 1059**: `getCombinedApprovalList` has a **massive commented-out block** (lines 547-603) that was attempting to "ENHANCE" data mapping — this was the original approach abandoned
5. **Lines 605-743**: The current `formattedData` mapping is the active code — it's complex and maps many fields from the API response
6. **Lines 851-862**: The approve button is **conditionally rendered**: `selectedRows.length > 0 && user.userRole !== 2` — role 2 (normal User) cannot approve
7. **Line 919**: The approve POST calls `/combined/approve` with `departmentID`, `taskEntryIDs`, `goalEntryIDs` — if backend changes the approval endpoint shape, this will break
8. **Line 1088-1104**: The reject POST calls `/combined/reject` with `id`, `type`, `comment` — if backend changes, this breaks
9. **Line 1091**: `departmentID` is conditionally included: `filterStatus ? Number(filterStatus) : null` — if filterStatus is empty string, may send `null` or `undefined`
10. **Line 1095**: `taskEntryIDs` filtered by `row.entryType === "Task" || row.entryType === "Target"` — only these types are included in task entries
11. **Line 1097**: `goalEntryIDs` filtered by `row.entryType === "Goal"` — only goals included
12. **Line 970-977**: Filter department Autocomplete only shows `ele.status === "Active"` departments — if backend changes status field, this breaks
13. **Line 986-1006**: Filter user Autocomplete has complex logic for finding user by key — may fail if user format changes
14. **Lines 1015-1029**: Filter entry type has hardcoded "Task"/"Target"/"Goal" options
15. **`pendingTasks` and `rejectedTasks` arrays** (lines 44-87) are **hardcoded sample data** — never used in actual API flow, dead code
16. **`getStatusChip` function** (lines 124-153) is defined but may not be used in the rendered table (the status renderCell in columns uses `getStatusChip` but the column chart may not call it correctly)
17. **Multiple `console.log` or `debugger` statements** left in the code

Backend Migration Impact

High risk — this page makes direct API calls to:

- `/combined/list` — **CRITICAL**: Returns combined data of tasks, targets, and goals. The response shape is complex with many nested fields. If Node.js backend changes field names, nesting, or pagination shape, the `formattedData` mapping (lines 605-743) will break.

- `/combined/approve` — Approval endpoint. Payload includes `departmentID`, `taskEntryIDs`, `goalEntryIDs`. If backend changes these field names or the response, approval will fail.
- `/combined/reject` — Rejection endpoint. Payload includes `id`, `type`, `comment`. If backend changes, rejection will fail.
- `/Department/list` — For department filter. Response mapped to department list.
- `/users/read` — For user filter. Response mapped to user list.

Critical concerns:

1. **`formattedData` mapping (lines 605-743)**: This is the most complex mapping in the codebase — it flat-maps `resData.data` entries, then maps each `entry.entries` item with many conditional fields. If the backend changes even one field name (e.g., `dailyTaskLogID` → `dailyTaskLogId`, or `goalID` → `goalId`), this entire mapping needs updating.

2. **Role-based access**:
   - `isELORUSER = [2, 3, 4, 5, 6, 7, 23].includes(user?.userRole)` — hardcoded role numbers
   - Approve button disabled for role 2 (normal User)
   - If backend changes role IDs, this will misbehave

3. **`departmentID` handling**:
   - Conditionally included in approve payload
   - If `filterStatus` is empty string `""`, `Number("")` is `0` — may send wrong department

4. **Entry type logic**:
   - `isTask = !!item.taskID`, `isTarget = !!item.targetID`, `isGoal = !!item.goalID`
   - If backend changes these field names, the type detection will fail

5. **Date range**:
   - `startDate/endDate` passed as `dateRange?.startDate || null`
   - If backend expects different date format, logs/approvals may fail

6. **Pagination**:
   - `page=0&size=0` is used in some calls (fetch first page, get totalRecords from response)
   - If backend changes pagination (e.g., page starts at 1, or size default changes), this may return wrong data

---

FormPage (src/pages/forms.tsx)

Route: `/department-forms`
Access: All authenticated users
File: `src/pages/forms.tsx` (369 lines)

Purpose & User Flow

Department form page — allows users to submit daily tasks and activity updates for their department. Features:

- Department and user filtering
- Task list display per department/user
- Daily activity log component for form submission

User flow:

1. User lands on page — defaults to their own department (based on `user.departmentID`)
2. Department and user can be filtered via autocomplete selectors
3. When department + user are selected, `DailyActivityLog` component renders
4. User can submit tasks via the DailyActivityLog
5. Task list can be fetched and displayed

API Calls

#	Method	Endpoint	Request Payload	Response Fields Used
1	`POST`	`/t2.0/authentificationUtilization/users/read?page=0&size=0`	`filter`, `searchText`, `departmentID`, `token`	User list: `userID`, `userName`, `roleID`
2	`POST`	`/t2.0/authentificationUtilization/Department/list?page=0&size=0`	`searchText`, `filter`, `startDate`, `endDate`, `roleID: user.userRole`, `userID: ...`	Department list: `departmentID`, `departmentName`, `departmentDescriptio` `newStatus`, `departmentStatus`
3	`POST`	`/t2.0/authentificationUtilization/departmentTasks/getTasksByDepartment`	`departmentId`, `roleId` (varies by role)	Tasks result: `taskID`, `taskName`, `description` `departmentID`, `standardTimeMins`, `frequency`, `isDynamic`, `isActive`, `modifiedAt`

4	POST	/t2.0/authentificationUtilization/departmentTasks/list?page=1&size=50	searchText, filter, startDate, endDate	All tasks: taskID, createdAt, taskName, description, departmentID, standardTimeMins, frequency, isDynamic, isActive, modifiedAt

Forms

Form	Fields	Validation	Defaults	Submit Behavior
Department filtering	department (autocomplete), user (autocomplete)	—	Defaults to user's own department	Updates filterStatus and selectedUserId, triggers task fetch
DailyActivityLog (imported component)	Task name, time taken, remarks, etc.	—	—	Submits daily task log

Tables/Lists

Table	Columns	Sorting	Filtering	Pagination	Export/ Download
Task list (from departmentTasks/getTasksByDepartment)	taskID, taskName, description, standardTimeMins, frequency, isActive	—	By department, by user	—	—
All tasks list (from departmentTasks/list)	taskID, createdAt, taskName, description, departmentID, standardTimeMins, frequency, isDynamic, isActive, modifiedAt	—	By department, user, search	—	—

Task columns (from getTasksByDepartment response):

- Task ID
- Task Name
- Description
- Standard Time (mins)
- Frequency (Daily/Weekly/Monthly/Dynamic)
- Is Dynamic (Yes/No)
- Is Active

All tasks columns (from `departmentTasks/list`):

- Task ID
- Created At
- Task Name
- Description
- Department ID
- Standard Time (mins)
- Frequency
- Is Dynamic
- Is Active
- Modified At

State

Type	Source	Description
Local state	Multiple `useState`	`sidebarOpen`, `selectedUserId`, `taskList`, `users`, `departmentList`, `filterStatus`, `departmentInput`
Derived	`getUserDepartments` function (lines 25-45)	Normalizes user departments — handles both `{ departmentID }` and `{ departments: [...] }` shapes
Effects	`useEffect` calls	Fetch users on mount (dependency: `[filterStatus]`); fetch departments on mount; fetch department tasks when `filterStatus` + `selectedUserId` change

---

Effects & Dependencies

- Line 64-82: `filterStatus` initializer — runs once; gets primary dept from user, or first dept from list
- Line 98-146: `fetchUsers` — dependency: `[filterStatus]` — fetches users for department filter
- Line 149-197: `fetchDepartments` — runs once on mount; handles multi-department users vs single dept users
- Line 199-222: `fetchDepartmentTask` — dependency: `[filterStatus, selectedUserId, users]` — fetches tasks by department/user
- Line 225-261: `fetchTasks` — dependency: `[filterStatus]` — fetches all tasks for the department

---

UI Components Used (tree)

```
FormPage
├── EmptyContainer (component) — when no filters selected and no tasks
```

```
├── Autocomplete (from @heroui/react) — department filter, user filter
├── AutocompleteItem (from @heroui/react) — autocomplete options
├── DailyActivityLog (component, ../components/DailyActivityLog) — main
├── Input (from @heroui/input) — department input (with search icon)
├── Button (from @heroui/button) — various actions
├── ReusableTable (component, ../components/table) — not directly used;
├── addToast (from @heroui/react) — notifications
├── Select, SelectItem (from @heroui/select) — department/user selector
├── useMemo (from react) — getUserDepartments normalization
├── format from date-fns — date formatting (in task creation)
├── title from primitives — page title
```

Known Defects, Dead Code, TODOs

1. **Lines 282-286**: `fetchDepartmentTask` body has `roleId: [1, 3, 4].includes(user.userRole) ? ... : user.userRole` — complex conditional role logic
2. **Line 204**: `departmentID: filterStatus ? Number(filterStatus) : 0` — if filterStatus is empty string, Number("") is 0, may send wrong dept ID
3. **Line 205**: `roleId` calculation is complex and may have edge cases
4. **Lines 98-146**: `fetchUsers` has `if (filterStatus !== null) return;` at top — means it only runs once when filterStatus is null, then never runs again
5. **Line 139-140**: `setSelectedUserId` logic: checks if current ID exists in mapped users, otherwise defaults to `mappedUsers.length ? mappedUsers[0].userID : user.userID` — may fail if user not in list
6. **Line 183**: `userID: (user.isHeExecutiveLeader) ? Number(user.userID) : Number(filterStatus)` — conditional based on executive leader status
7. **Line 196**: `if (departmentList.length > 0) { setFilterStatus(departmentList[0].key); ... }` — auto-selects first department, may surprise users
8. **`tasks` state** (line 89): `const [tasks, setTasks] = useState<Task[]>([])` — initialized as empty type-annotated array
9. **Commented-out `fetchDepartmentTask` effect** (lines 258-324 in the original exploration, but in this file lines 199-222 is the active one, earlier version had commented out second fetch)
10. **No clear submit action** for the daily activity log — the `DailyActivityLog` component handles its own submission, but there's no visible "Submit" button on this page
11. **Line 352-357**: `DailyActivityLog` rendered with `DeptId={filterStatus} selectedUserId={selectedUserId} selectedUserRole=...` — passes department/user info down

---

Backend Migration Impact

Medium risk — this page makes API calls to:

- `/users/read` — user list for filters

- `/Department/list` — department list for filters
- `/departmentTasks/getTasksByDepartment` — tasks by department + role
- `/departmentTasks/list` — all tasks with pagination

If Node.js backend changes:

- **User response**: `userID`, `userName`, `roleID` field names — if changed, user filter breaks
- **Department response**: `departmentID`, `departmentName`, `newStatus`, `departmentStatus` — if changed, department filter breaks
- **Tasks response**: Field names like `taskID`, `taskName`, `description`, `standardTimeMins`, `frequency`, `isDynamic`, `isActive`, `modifiedAt` — if changed, task lists break
- **Role-based logic**: `[1, 3, 4].includes(user.userRole)` — if backend changes role IDs, the role-based task fetching will break
- **Default department ID `0`**: `Number(filterStatus) || 0` — if backend changes how departments are numbered, may fetch wrong tasks

Additional concerns:

- `getUserDepartments` normalization (lines 25-45): Handles both `{ departmentID }` and `{ departments: [...] }` user shapes. If backend changes one of these shapes, the normalization will produce incorrect results
- `filterStatus` default of `0` vs `""` — inconsistent behavior
- The page auto-selects the first department on mount (line 196) — may not be the user's expected department if backend changes department ordering

---

GoalManagement Page (src/pages/goalManagement.tsx)

Route: `/Goal-management`
Access: `adminOrTech` guard (requires `user.userRole === 1` OR `user.departmentID === 24`)
File: `src/pages/goalManagement.tsx` (711 lines)

Purpose & User Flow

Goal management page — allows admins/tech to manage department goals. Features:

- View goals list with filtering by department
- Add new goals
- Edit existing goals
- Toggle goal status (activate/deactivate)
- Filter by frequency and status
- Download all goals

User flow:

1. Admin/Tech views goals list for their department
2. Clicks "Add New Goal" → opens modal
3. Fills goal name, metric, metric value, unit, frequency, timing type, description
4. Submits → creates goal, refreshes list
5. Clicks edit on a goal → opens edit modal
6. Updates fields → saves changes
7. Clicks activate/deactivate → toggles goal status

API Calls

#	Method	Endpoint	Request Payload	Response Fields Used	When
1	POST	/t2.0/authentificationUtilization/Department/list?page=0&size=0	searchText, filter, startDate, endDate, roleID: 1, userID: 1	Department list: departmentID, departmentName, description, status	On mou (once)
2	POST	/t2.0/authentificationUtilization/departmentGoals/list?page=&size=	departmentId: 0, searchText, filter, startDate, endDate	Goals list: goalID, departmentID, goalName, goalMetric, metricValue, unit, frequency, isDynamic, isActive, liveAccounts, roleID	When pageNu pageSi search changes
3	POST	/t2.0/authentificationUtilization/departmentGoals/create	departmentID, goalName, goalMetric, metricValue, unit, liveAccounts, frequency, timingType, isDynamic, description, roleID	Success toast	When " Goal" submitt
4	PUT	/t2.0/authentificationUtilization/departmentGoals/update/${goalID}	Same as create payload	Success toast	When " Goal" submitt

5	PUT	/t2.0/authentificationUtilization/departmentGoals/${row.goalID}/status?isActive=${newStatus}	{}	Success toast	When "Activat Deactiv submitt

Forms

Form	Fields	Validation	Defaults	Submit Behavior
Add/Edit Goal Modal	goalName (required), goalMetric (required), metricValue (required), unit (number, required), frequency (select), timingType (select — Ongoing/ Fixed), liveAccounts (conditionally required when filterStatus = = = 12), roleID (select), description (optional)	All fields required except description	goalId: -1 (new), frequency: "Monthly", timingType: "MonthEnd", isDynamic: false, isActive: true, roleID: ""	On submit: creates or updates goal, refreshes list, resets form
Toggle Status Confirmation	—	—	—	On submit: updates goal status via API

Tables/Lists

Table	Columns	Sorting	Filtering	Pagination	Export/Download
Goals table	Department ID, Goal Name, Metric, Unit, Frequency, Dynamic, Status	✓ (via ReusableTable)	✓ (department filter, frequency options, status options)	✓ (rows per page)	✓ (Download All button, visible only when no department filter or userID = = = 1 or dept = = = 16)

Goal columns (from `goalColumns`, lines 187-195):

- Department ID
- Goal Name
- Metric
- Unit
- Frequency
- Dynamic
- Status

Filter options (from `filterOptions`, lines 478-499):

- Frequency: Daily, Weekly, Monthly, Dynamic, Not Daily
- Status: Active, Inactive

Table actions (from `taskActions`, lines 200-228):

- Edit: opens goal in modal for editing
- Activate/Deactivate: toggles goal status (requires exactly one row selected)

Download button (lines 381-387):

- Visible only when `!authUserLD?.departmentID` (no dept filter) OR `authUserLD?.userID === 1` OR user's departments include deptID 16
- Disabled when `loading || !!authUserLD?.departmentID`
- Downloads all goals

State

Type	Source	Description
Local state	Multiple `useState`	`sidebarOpen`, `selectedDepartment`, `departmentList`, `SelectedTask`, `loading`, `isEditing`, `selectedRows`, `confirmOpen`, `toggleStatusValue`, `tableRefreshKey`, `pageNumber`, `totalRecords`, `pageSize`, `searchValue`, `goals`, `isModalOpen`, `newGoal`
Derived	`useMemo`	—
Effects	Multiple `useEffect`	Fetch departments on mount; fetch goals when `pageNumber`, `pageSize`, `searchValue`, or `tableRefreshKey` change; handle toggle status; handle edit/add

---

Effects & Dependencies

- Line 230-269: `fetchDepartments` — runs once on mount (hardcoded `roleID: 1`, `userID: 1`)
- Line 272-316: `fetchGoals` — dependency: `[pageNumber, pageSize, searchValue, tableRefreshKey]` — fetches goals with pagination and search
- Line 318-362: `handleToggleStatus` — on confirm, updates goal status via API, refreshes goals list
- Line 88-96: `resetGoalForm` — resets form state and filterStatus

UI Components Used (tree)

```
GoalManagement
├── ReusableTable (component, ../components/table) — goals table with f
├── Modal (from @heroui/react) — add/edit goal modal, status confirmati
├── ModalContent, ModalHeader, ModalBody, ModalFooter (from @heroui/rea
```

```
├── useDisclosure (from @heroui/react) — for modals
├── Input (from @heroui/input) — goal name, metric value, unit, live ac
├── Select, SelectItem (from @heroui/select) — frequency, role, timing
├── Button (from @heroui/button) — add goal, activate, deactivate, canc
├── Tabs, Tab (from @heroui/react) — not actually used in this componen
├── addToast (from @heroui/react) — notifications
├── Textarea (from @heroui/input) — description (commented out, not ren
├── ReusableTable filterOptions — frequency and status filter arrays
├── useMemo, useCallback (from react) — state management
├── format from date-fns — date formatting (in download payload)
├── title from primitives — page title
├── Goal (interface, lines 14-28) — goal shape definition
```

Known Defects, Dead Code, TODOs

1. **Lines 608-617**: `Live Accounts` input is conditionally rendered only when `filterStatus == 12` — hardcoded department ID check
2. **Lines 621-631**: `Role` select has hardcoded role IDs: `1` = Root User, `2` = User, `3` = EL, `4` = FL, `5` = Intern, `6` = Executive, `7` = Manager, `23` = Lead — if backend changes role IDs, this breaks
3. **Lines 633-643**: `Timing Type` select has only 2 options (Ongoing, Fixed) — commented out 3rd option section
4. **Lines 645-652**: `Is Dynamic` select has true/false options — commented out section
5. **Lines 645-681**: `Description` Textarea is **commented out** — not rendered in the form
6. **Line 76**: `newGoal` initial state has `goalId: -1` — magic number used as sentinel for "new" goal
7. **Line 85**: `resetGoalForm` also resets `filterStatus` to "" — may cause unexpected department filter reset
8. **Line 238**: `fetchDepartments` uses hardcoded `roleID: 1`, `userID: 1` — always fetches depts for root user, may not be correct for all logged-in users
9. **Line 288**: `body.departmentId: 0` in fetchGoals — always fetches goals for dept 0, may need to be `filterStatus`
10. **Lines 492-501**: Commented out `efficiencyLevel` calculation at bottom of file — same efficiency logic as in dashboard/efficiency pages
11. **Multiple `console.log` statements** in the code (e.g., line 293: `console.log("departmentlist", formatted)`)
12. **Download button visibility logic** (lines 381-387): Complex condition `!authUserLD?.departmentID || authUserLD?.userID === 1 || (user?.departments ?? []).some((d) => Number(d?.departmentID) === 16)` — hardcoded department ID 16
13. **`isEditing` state** management: When editing, form fields are populated from `newGoal` which is also used for adding — potential for state corruption

---

Backend Migration Impact

High risk — this page makes direct API calls to:

- `/Department/list` — for department filter
- `/departmentGoals/list` — **CRITICAL**: Goals list with pagination, search, and filtering
- `/departmentGoals/create` — Goal creation
- `/departmentGoals/update/${goalID}` — Goal update
- `/departmentGoals/${goalID}/status` — Goal status toggle

If Node.js backend changes:

- **Goals response**: The `departmentGoals/list` response shape includes `response.goals` array with fields: `goalID`, `departmentID`, `goalName`, `goalMetric`, `metricValue`, `unit`, `frequency`, `isDynamic`, `timingType`, `isActive`, `liveAccounts`, `roleID`. If any field names change, the mapping (lines 294-309) will break.
- **Live Accounts**: Conditionally required only when `filterStatus === 12` — if backend changes which department triggers this, or the field name `liveAccounts`, the form validation will break.
- **Role IDs**: Hardcoded in the Role select (lines 598-606): `1` = Root User, `2` = User, etc. If backend changes role IDs, the form will show wrong roles.
- **Frequency options**: Hardcoded (Daily, Weekly, Monthly, Quarterly, Dynamic, Not Daily) — if backend changes frequency values, the form will have wrong options.
- **Timing Type**: Currently only Ongoing/Fixed shown — if backend adds more timing types, UI won't reflect them (commented out section exists).
- **Status toggle**: `/departmentGoals/${row.goalID}/status?isActive=${newStatus}` — if backend changes the status endpoint format, toggle will fail.
- **Download logic**: The condition checking `userID === 1` or `departmentID === 16` is hardcoded — if backend changes which users/get access to "Report" feature, this needs updating.

Additional concerns:

- `fetchDepartments` hardcodes `roleID: 1`, `userID: 1` — always fetches for root user, may not reflect the logged-in user's departments
- `body.departmentId: 0` in fetchGoals — always fetches for dept 0; should probably use `filterStatus`
- The `filterStatus` state defaults to `""` and is set from department list on mount — if department list is empty, no filter applied
- Role-based access: `adminOrTech` guard on route, but internal logic also checks `isEditing`, `toggleStatusValue`, etc. — consistency needed if roles change

---

Goals Report Page (src/pages/goals.tsx)

Route: `/goals`
Access: All authenticated users
File: `src/pages/goals.tsx` (899 lines)

Purpose & User Flow

Goals report page — tracks goal progress and performance over time. Features:

- Date range picker (7/30 days or custom)
- Department and user filtering
- Summary card showing average goal achieved %
- ReusableTable displaying goal data with various columns
- EL (Executive Leader) filter for role 4 users
- Department filtering for role 4 users

User flow:

1. User lands on page with default 7-day range
2. User can filter by department (shows for all roles) and user (filtered by role)
3. User can select "Target" or "Task" filter
4. Summary card shows average goal achieved percentage
5. Table displays goal data with work dates, user names, goals, achieved values
6. User can change date range, filters, or switch between target/task views

---

API Calls

#	Method	Endpoint	Request Payload	Response
1	POST	/t2.0/authentificationUtilization/Department/list?page=0&size=	searchText, filter, startDate, endDate, roleID: 3, userID: ...	Department list departmentII departmentNa executiveLea functionalLe
2	POST	/t2.0/authentificationUtilization/users/read?page=0&size=0	filter, searchText, departmentID, ... (filterStatus ? {} : null)	User list: userI roleID
3	POST	/t2.0/authentificationUtilization/userGoalData/list?page=&size=	departmentID, userID, startDate, endDate, goalStartDate, goalEndDate, roleID, eLID, isHeExecutiveLeader, searchText	userGoalData userGoalData userID, userN departmentII goalName, fix achievedGoal goalEfficien recalculated

4	Conditional POST	/t2.0/ authentificationUtilization/ list-efficiency-summary	departmentID, startDate, endDate, userID, groupBy: "day", isHeExecutiveLeader, efficiencyLevel	summary.Avg data array	

Special Logic

Logic	Description
showRecalculatedTarget (line 101)	Number(user?.departmentID) === 21 — shows fixedGoal differently for department 21
shouldCallEfficiency (lines 530-533)	Calls efficiency summary ONLY when: dept===7 AND user===90 AND date within cutoff (2026-06-30)
Two data fetching paths:	1) If condition above met → use efficiency summary API 2) Otherwise → use userGoalData API
isWithinCutoff (lines 526-528)	dateRange.startDate <= "2026-06-30" && dateRange.endDate <= "2026-06-30" — hardcoded cutoff date
averageGoalAchieved (line 644)	From res.data.response.avgPercent when using userGoalData API

Forms

- None — pure display/filter page with table and summary card

---

Tables/Lists

Table	Columns	Sorting	Filtering	Pagination	Export/ Download
Goals report table	userGoalDataID, User ID (conditional), Work Date, User Name, Goal Name, Fixed Goal, Achieved, Comments	✓ (via ReusableTable)	✓ (department filter, user filter, EL filter, entry type filter)	✓ (rows per page)	—

reportColumns (lines 124-240):

- userGoalDataID (always shown but labeled "userGoalDataID")
- User ID: shown only when Number(filterStatus) !== 7 (conditional)
- Work Date: rendered with format(new Date(item.workDate), "dd MMM yyyy")

- User Name
- Goal Name
- Fixed Goal: **special logic** — if `departmentID === 21` show `fixedGoal`, if `departmentID === 37` show `fixedGoal` (same), otherwise just `fixedGoal`
- Achieved: shown as number or "0" (text-red-500) if 0
- Comments: shown or "—"

Special column logic:

- Fixed Goal cell renderer (lines 159-182): Different handling based on `item.departmentID` (21 or 37)
- Achieved cell renderer (lines 186-197): Shows "0" in red if achievedGoal is 0

Filter options (various DropdownItems in the filter dropdown):

- EL selector (for role 4 users): shows users where `user.userID === user.reportingManagerID` or `u.roleID === 3`
- Department filter: shows departments with role-based filtering
- Entry type: ALL/TARGET/TASK (commented out in the read, but UI has filter)
- User filter: shows all users or filtered by EL role

State

Type	Source	Description
Local state	Multiple `useState`	`isTarget`, `taskIDFilter`, `averageGoalAchieved`, `reportData`, `pageNumber`, `pageSize`, `totalRecords`, `loading`, `departmentList`, `users`, `ELList`, `elFilter`, `isFilterOpen`, `searchValue`, `dateSelect`, `dateRange`, `showRecalculatedTarget`
Derived	`useMemo`	`filteredUsers` from `users` + `userSearch`
Effects	Multiple `useEffect`	Fetch departments when `elFilter` changes; fetch users when `filterStatus` changes; fetch data when `filterStatus`, `dateRange`, `selectedUser`, `pageNumber`, `pageSize`, or `elFilter` changes; hardcoded `CUTOFF_DATE` logic

---

Effects & Dependencies

- Line 246-287: `fetchDepartments` — dependency: `[elFilter]` — fetches departments with roleID 3 and userID based on elFilter
- Line 289-328: `fetchUsers` — dependency: `[filterStatus]` — fetches users; conditional: if filterStatus set → setUsers, else → setELList
- Line 408-657: `fetchData` — **main data fetch**; dependency: `[filterStatus, dateRange, selectedUser, pageNumber, pageSize, elFilter]` — complex logic with two API paths (efficiency summary vs userGoalData)
- Line 655: `fetchData()` called on every dependency change

UI Components Used (tree)

```
GoalsReport
├── ReusableTable (component, ../components/table) — main goal data tab
├── Dropdown, DropdownMenu, DropdownTrigger, DropdownItem (from @heroui
├── Button (from @heroui/button) — various actions, filter dropdown tri
├── DateRangePicker (from @heroui/react) — date range selector
├── CustomDateRangePicker (component, ../components/CustomDateRangePick
├── FilterIcon (inline SVG) — filter icon in dropdown
├── Tabs, Tab (from @heroui/react) — not directly used in rendering but
├── Add Filter dropdown item (conditional, lines 750-802) — sub-departm
├── Autocomplete, AutocompleteItem (from @heroui/react) — EL select, de
├── Format from @internationalized/date and date-fns — date formatting
├── Filter from @heroui/react — various filter autocompletes
├── Select, SelectItem (from @heroui/select) — department, user, EL fil
├── today, getLocalTimeZone from @internationalized/date — timezone han
├── addToast (from @heroui/react) — notifications
├── title from primitives — page title
```

Known Defects, Dead Code, TODOs

1. **Line 101**: `showRecalculatedTarget = Number(user?.departmentID) === 21` — hardcoded department ID 21 check
2. **Lines 526-528**: `CUTOFF_DATE = "2026-06-30"` — hardcoded date; if current date passes this, efficiency path won't be used
3. **Lines 530-533**: `shouldCallEfficiency` condition: `Number(filterStatus) === 7 && Number(selectedUser) === 90 && isWithinCutoff` — very specific hardcoded conditions (dept 7, user 90)
4. **Lines 330-373**: Commented-out `fetchGoals` effect — dead code with alternative API calling approach
5. **Lines 380-406**: Commented-out alternative fetch that calls BOTH efficiency AND userGoalData APIs in parallel, then merges data
6. **Lines 520-525**: `efficiencyData` mapping creates `achievedGoal: 100` and `goalEfficiencyPercent: (achievedGoal / fixedGoal) * 100` — frontend calculation that may disagree with backend
7. **Lines 613-615**: `setaverageGoalAchieved(res.data.response.avgPercent)` — gets avg percent from userGoalData response, but if using efficiency path, uses `summary.AvgTargetPercent`
8. **`filterStatus` default** (line 76-78): `user?.departmentID ? String(user.departmentID) : ""` — conditional based on user having department
9. **`selectedUser` default** (lines 80-82): `user?.userID ? user.userID == 1 ? null : String(user.userID) : ""` — special case for userID 1
10. **`elFilter` default** (lines 84-86): `user?.isHeExecutiveLeader ? Number(user.userID) : 0` — if executive leader, use own userID; else 0

11. **Complex conditional rendering** throughout: `user.userRole !== 2 && user.userRole !== 5 && user.userRole !== 6 && user.userRole !== 7 && user.userRole !== 23` — excludes certain roles from showing filter dropdown
12. **Department filter logic** (lines 780-799): Complex role-based filtering of departments — if `user.userRole === 4` (FL), only shows depts where `ele.functionalLeaderID == user.userID`; otherwise shows active depts
13. **`showRecalculatedTarget`** affects the `fixedGoal` render cell (line 160): Only for dept 21 and 37 — other departments just show `fixedGoal` without the conditional
14. **Multiple `console.log` statements** left in the code
15. **`recalculatedTarget` field** from API response used in some paths but not others — inconsistent data usage

---

Backend Migration Impact

High risk — this page makes direct API calls to:

- `/Department/list` — for department filter (with roleID 3 hardcoded)
- `/users/read` — for user list (conditional based on filterStatus)
- `/userGoalData/list` — **CRITICAL**: Primary data source for goal progress

Two API paths exist:

Path 1 (Efficiency Summary) — only when `Number(filterStatus) === 7 && Number(selectedUser) === 90 && isWithinCutoff`:

- Endpoint: `/t2.0/authentificationUtilization/list-efficiency-summary`
- Payload: `departmentID`, `startDate`, `endDate`, `userID`, `groupBy: "day"`, `isHeExecutiveLeader`, `efficiencyLevel`
- Response used: `effRes.data.data` mapped to `formatted` with fields: `userGoalDataID`, `workDate`, `userID`, `userName`, `departmentID`, `goalID: 0`, `goalName: "TAT"`, `fixedGoal: 100`, `achievedGoal: achievedGoal`, `goalEfficiencyPercent: (achievedGoal / fixedGoal) * 100`, `recalculatedTarget: null`
- `averageGoalAchieved = summary.AvgTargetPercent`
- **Risk**: This is a very specific path (dept 7, user 90, date before 2026-06-30). If Node.js changes this endpoint or the response shape, this path breaks.

Path 2 (User Goal Data) — default path:

- Endpoint: `/t2.0/authentificationUtilization/userGoalData/list?page=&size=`
- Payload includes: `departmentID`, `userID`, `startDate`, `endDate`, `goalStartDate`, `goalEndDate`, `roleID`, `eLID`, `isHeExecutiveLeader`, `searchText`
- Response used: `res.data.response.userGoalDataList` mapped with many fields including `comment: item.comment || ""`
- `averageGoalAchieved = res.data.response.avgPercent`
- **Risk**: The response shape is complex with many conditional fields. If backend changes field names (e.g., `goalEfficiencyPercent` → `goalEfficiencyPercent`,

`recalculatedTarget` structure, `fixedGoal` handling), the mapping (lines 626-640) will break.

Critical concerns:

1. **`userGoalData/list` response mapping** (lines 626-640): Maps many fields from the API response including `userGoalDataID`, `workDate`, `userID`, `userName`, `departmentID`, `goalID`, `goalName`, `fixedGoal`, `achievedGoal`, `goalEfficiencyPercent`, `recalculatedTarget`, `comment`. This is the most critical mapping — any field rename breaks the table.

2. **Department ID 21 special handling** (line 101, 160-166): `showRecalculatedTarget` and the fixedGoal render cell both check `item.departmentID === 21`. If backend changes department IDs or the way department 21 is identified, this special handling will misbehave.

3. **Hardcoded cutoff date** `2026-06-30` (lines 526-528): This is a future date relative to the current date (2026). When this date passes, the efficiency summary path will never be used, and the userGoalData path will always be used instead. This needs to be updated to a rolling date or removed.

4. **Role-based filtering** throughout: `isELORUSER`, department filtering by role, user filtering by role — all use hardcoded role numbers `[2, 3, 4, 5, 6, 7, 23]` and conditional logic. If backend changes role IDs, all this filtering will break incorrectly.

5. **`selectedUser` special case for userID 1** (line 81-82): `user.userID == 1 ? null : String(user.userID)` — user 1 gets null selectedUser, which changes behavior. If backend changes user 1's ID or the special-casing logic, this breaks.

6. **`elFilter` logic** (lines 84-86, 254-258): `user?.isHeExecutiveLeader ? Number(user.userID) : 0` — if executive leader, filter by own ID; else 0. If backend changes `isHeExecutiveLeader` flag or its meaning, this breaks.

History Page (src/pages/History.tsx)

Route: `/User-History`
Access: `adminOnly` guard (requires `user.userRole === 1`)
File: `src/pages/History.tsx` (1044 lines)

Purpose & User Flow

User performance history page — reviews historical performance, productivity, utilization, and target achievement trends. Features:

- Department and user filtering via filter dropdown

- Summary cards showing avg target, utilization, goal percentages per department
- Department detail cards with efficiency levels
- Drill-down modal to view targets and goals per department
- Download efficiency summary Excel (button commented out)
- Executive leader summary tabs (for `isHeExecutiveLeader` users)

User flow:

1. Admin lands on page — filter dropdown allows user selection
2. Summary cards show department-level efficiency metrics (only when user selected)
3. User can click "View Details" on a department card
4. Modal opens showing department targets (🎯) and goals (🚀)
5. EfficiencyTable shows raw data (hidden for role 3)
6. EL users see tabbed department summaries

API Calls

#	Method	Endpoint	Request Payload	Response Fields Used
1	POST	/Department/list?page=0&size=0	searchText, filter, startDate, endDate, roleID: user.userRole, userID	response.department departmentID, departmentName, departmentDescripti newStatus
2	POST	/users/read?page=0&size=0	filter, searchText, departmentID: filterStatus ? Number(filterStatus) : 0	response.users[]: userID, userName
3	POST	/list-efficiency-summary	departmentID, startDate, endDate, userID, groupBy: "day", history: true, isHeExecutiveLeader, efficiencyLevel	summary object keyed by departmentID; data[] w DepartmentID, DepartmentName, TargetAchievedPerce AllocatedTimeHrs, UtilizationPercent, TotalTimeOfWork, GoalPercent, WorkDat departments[] for EL summary
4	POST	/departmentTargets/getTargetsByDepartment	departmentId: dept.DepartmentID	result[]: targetID, targetName, unit
5	POST	/departmentGoals/getGoalsByDepartment	departmentId: dept.DepartmentID	result[]: goalID, goalName, frequency, unit

6	POST	/download-efficiency-summary	Same as #3 (minus history)	Excel blob (spreadsheetml.sheet

---

Special Logic

Logic	Description
isELORUSER (line 84)	user.userRole === 2 || user.userRole === 3 || user.userRole === 4 — if true, selectedUser auto-set to user's own ID
Efficiency level calc (lines 471-485)	High: AvgTargetPercent >= 100 AND AvgUtilizationPercent >= 80 AND AvgTotalWork <= AvgAllocatedHrs; At Par: >= 80 / >= 50 / <= allocated; else Low
Department summary reduction (lines 434-500)	Reduces data[] into per-department summary objects with computed averages and efficiency levels
handleViewDetails (lines 241-269)	Fetches targets + goals for a specific department when modal opened
Date range default (lines 90-123)	If selectedUser set → clears date range to empty strings; otherwise default 7 days from today
Filter dropdown visibility (line 644)	user.userRole !== 2 — role 2 users don't see Filter button
EfficiencyTable hidden for role 3 (line 842)	user.userRole !== 3 — executive leaders don't see main table
EL summary tabs (line 856)	user.isHeExecutiveLeader — shows tabbed EfficiencyTable per department from data.departments
history: true flag (line 409)	Sent only in dashboard fetch, not in download — unique to this page

---

State

Type	Source	Description
Local state	Multiple useState	sidebarOpen, dateSelect, isModalOpen, selectedDept, targets, goals, departmentList, users, pageNumber, totalRecords, pageSize, searchValue, average, ELUserSummary, dashboard, loading, dateRange, filterStatus, efficiencyFilter, isFilterOpen, selectedUser, userSearch, reportData
Derived	Computed	filteredUsers — users filtered by userSearch

Auth	`localStorage.authUserLD`	Parsed once; `user` object used throughout

Effects & Dependencies

- Lines 102-123: Date range effect — dependency: `[selectedUser]` — clears or sets 7-day default
- Lines 271-308: `fetchDepartments` — runs once on mount; uses `fetch()` (not axios) with `roleID: user.userRole`
- Lines 310-349: `fetchUsers` — dependency: `[filterStatus]` — uses axios with Bearer token
- Lines 399-521: `dashboard` — **main data fetch**; dependency: `[filterStatus, dateRange, selectedUser, efficiencyFilter]` — fetches efficiency summary, computes department summary, sets `average` (only if `selectedUser` set) and `dashboard`, and `ELUserSummary` for role 3
- Lines 524-573: `handleDownload` — fetches Excel blob, triggers download via anchor element

---

Dead Code

- Lines 351-397: Entire `fetchLogs` effect **commented out** (dailyTaskLog/list endpoint)
- Lines 574-583: Efficiency level ternary **commented out**
- Lines 600-641: Download button and date range dropdown **commented out** in JSX
- Lines 660-686: Department filter autocomplete **commented out** in filter dropdown
- Lines 722-738: Efficiency filter autocomplete **commented out** in filter dropdown
- Lines 1009-1044: Fallback target/goal fetch logic **commented out** at file end
- Lines 169-186: `reportActions` with "View Details" and "Edit" — both have empty `onAction` callbacks (no-op)

UI Components Used (tree)

```
History
├── EfficiencyTable (component, ../components/table2) — main data table
├── Card, CardBody, CardHeader (from @heroui/react) — summary cards + m
├── Modal, ModalContent, ModalHeader, ModalBody (from @heroui/react) —
├── Tabs, Tab (from @heroui/react) — EL department summary tabs
├── Dropdown, DropdownMenu, DropdownTrigger, DropdownItem (from @heroui
├── Autocomplete, AutocompleteItem (from @heroui/react) — user filter
├── Chip (from @heroui/react) — efficiency level badge, target/goal chi
├── Spinner (from @heroui/react) — loading state
├── Button (from @heroui/react) — Close button in filter, (Download com
├── CustomDateRangePicker (component, ../components/CustomDateRangePick
├── FilterIcon (inline SVG, lines 47–62) — filter button icon
├── title (from ../components/primitives) — page title
├── format (from date-fns) — (imported but only used in commented code)
```

```
├── Sidebar, Topbar (imported but not rendered in JSX)
```

Known Defects, Dead Code, TODOs

1. **Lines 63-64 (previous file)**: Previous version had garbled content — now fixed
2. **Lines 351-397**: Large commented-out `fetchLogs` effect — dead code referencing `/dailyTaskLog/list`
3. **Lines 600-641**: Download button and date range picker both commented out — `handleDownload` function exists but no button calls it (dead code)
4. **Lines 660-686, 722-738**: Department and efficiency filters commented out — only user filter remains active in dropdown
5. **Lines 1009-1044**: Commented-out fallback logic at file end
6. **Lines 169-186**: `reportActions` has empty callbacks — `View Details` and `Edit` buttons do nothing
7. **`filterStatus` initialization** (line 128-130): `user?.departments ? 0 : ""` — mixes number (0) and string ("") types
8. **`isELORUSER`** (line 84): Only checks roles 2, 3, 4 — differs from other pages that check `[2,3,4,5,6,7,23]`
9. **Role 3 special handling** (line 511): `if (user.userRole == 3) { setELUserSummary(data.departments) }` — only role 3 gets EL summary; `==` instead of `===`
10. **Line 644**: `user.userRole !== 2` for filter visibility — hardcoded role check
11. **Line 842**: `user.userRole !== 3` for EfficiencyTable visibility — hardcoded role check
12. **`user.departmentID` vs `filterStatus`**: Department filter autocomplete is commented out but `filterStatus` still used in API calls — always passes `0` (from initial value)
13. **`Sidebar` and `Topbar` imported but not rendered** — dead imports
14. **Multiple `console.log` statements** left in code
15. **`reportData` state** (line 124) set only in commented-out effect — always empty array

---

Backend Migration Impact

High risk — multiple API endpoints with unique request flags:

Key concerns:

1. **`history: true` flag** (line 409): Unique to this page's `/list-efficiency-summary` call — if backend doesn't recognize this flag, history page may return wrong data
2. **Efficiency level computation** (lines 471-485): Duplicated frontend business logic — same thresholds as 04-efficiency, 14-query-report, 15-reports. If backend changes thresholds, 4+ pages show wrong levels
3. **`departmentTargets/getTargetsByDepartment`** and **`departmentGoals/getGoalsByDepartment`**: Modal drill-down endpoints — response `result[]` with `targetID/targetName/unit` and `goalID/goalName/frequency/unit` field

names
4. **Department summary reduction** (lines 434-500): Complex frontend aggregation of `data[]` into per-department summaries — if backend field names (`TargetAchievedPercent`, `AllocatedTimeHrs`, `UtilizationPercent`, `TotalTimeOfWork`, `GoalPercent`) change, summary cards break
5. **EL summary structure** (line 513): `data.departments` must contain `{ DepartmentName, Users[] }` — if backend changes this shape, EL tabs break
6. **Hardcoded role checks**: `userRole !== 2`, `userRole !== 3`, `userRole === 3`, `isELORUSER` roles 2/3/4 — all break if role IDs change
7. **`user.isHeExecutiveLeader`**: Boolean flag from auth object — if backend changes this field name or type, EL tabs disappear
8. **Mixed fetch/axios**: Uses `fetch()` for departments (line 283) and `axios` for everything else — inconsistent error handling

---

Access Management Page (src/pages/access-management.tsx)

Route: `/access-management`
Access: `adminOnly` guard (requires `user.userRole === 1`)
File: `src/pages/access-management.tsx` (959 lines)

Purpose & User Flow

Access management — admin users can manage all users in the system: view list, add new users, edit user details, toggle user status (active/inactive), delete users, and manage role permissions. Features:

- User list with filtering by role, status, department
- Add new user modal with full form
- Edit user modal (when selecting a row)
- Toggle user status confirmation
- Delete user confirmation
- Role access modal (configure permissions per module)
- Department list for filtering

User flow:

1. Admin views user list
2. Can filter by role, status, or department
3. Selects a user row → can edit, toggle status, or delete
4. "Add New User" button → opens form modal
5. Form has department, role, reporting manager, EL, appraisal cycle, feedback resolver team

6. "Role Access" button → opens permission configuration modal
7. Department/user filters update the user list

API Calls

#	Method	Endpoint	Request Payload	Response Field
1	POST	/t2.0/authentificationUtilization/Department/list?page=0&size=0	searchText, filter, startDate, endDate, roleID: 1, userID: 1	Department list: departmentID, departmentNam description, newStatus
2	POST	/t2.0/authentificationUtilization/users/read?page=&size=	filter: statusFilter, searchText: searchValue, departmentID: null, roleID: Number(roleFilter)	User list with full formatting: user employeeID, us userEmail, departmentNam departmentID, role, secondar newStatus, isA userCreateTim reportingMana reportingMana elManagerId, elManagerName appraisalCycl feedbackResol
3	POST	/t2.0/authentificationUtilization/users/read?page=0&size=0	filter: "Active", searchText: searchValue, departmentID: null, roleID: Number(roleFilter)	All users without pagination → stor usersAll and setAllUser
4	POST	/t2.0/authentificationUtilization/usersArchive	userIdList: [deleteConfirm.id]	Success toast
5	POST	/t2.0/authentificationUtilization/updateUserStatus	`userStatus: 1	0, userIdList: [user.userID]`

6	POST	/t2.0/authentificationUtilization/User/create	Full user payload: employeeID, userName, userEmail, password, departmentID, role, roleID, secondaryRoleID, reportingManagerId, elManagerId, appraisalCycle, feedbackresolverteam	Success toast
7	PUT	/t2.0/authentificationUtilization/User/update	Full user payload (same as create)	Success toast

Forms

Form	Fields	Validation	Defaults	Submit Behavior
Add/Edit User Modal	Employee ID (required), Full Name (required), Email (required, type = email), Password (conditionally required), Department (required, autocomplete), Role (required, select), Secondary Role (select), Reporting Manager (assessor 1, autocomplete), EL / Dept Head (assessor 2, autocomplete), Appraisal Cycle (select), Feedback Resolver Team (select)	All fields required except secondary role; password required for create only (commented out)	New user: employeeID:"", name:"", email:"", password:"", department:"", role:"User", secondaryRole:"", feedbackResolverTeam:""	On submit: creates or updates user, refreshes list, resets form, reloads page
Role Access Modal	Permission configuration per module (tiers: basic/createOnly/advanced/root/custom)	—	—	On submit: saves role privileges via API

Form Field Details

Password field (lines 718-730): Commented out `!isEditMode && (...)` — password is only shown for new users, not edits.

Department Autocomplete (lines 731-756):

- `isRequired={newUser.role !== "Root User"}` — Root User doesn't need department
- Special handling: `prev.role === "Root User" ? null : key ? String(key) : null` — sends null for Root User department

Reporting Manager EL Autocomplete (lines 757-789):

- Pulls from `usersAll` (all users without pagination)
- `isRequired={newUser.role !== "Root User"}`

EL / Dept Head Autocomplete (lines 789-809):

- Same source `usersAll`
- `isRequired={newUser.role !== "Root User"}`

Appraisal Cycle (lines 807-817):

- Two options: "April Cycle (Apr–Mar)" and "October Cycle (Oct–Sep)"

Role Select (lines 818-834):

- Options: Root User, EL, FL, User, Intern, Executive, Manager, Lead
- Maps to roleIDs: 1, 3, 4, 2, 5, 6, 7, 23 respectively

Secondary Role Select (lines 835-854):

- Same options as Role, but optional

Feedback Resolver Team (lines 855-870):

- Options: HR, L&D

Tables/Lists

Table	Columns	Sorting	Filtering	Pagination	Export/ Download
User list table	ID, Employee ID, Name, Email, Department, Reporting Manager, EL Manager, Appraisal Cycle, Role, Status	✓ (via ReusableTable)	✓ (roleID, newStatus filter options)	✓ (rows per page)	—

userColumns (lines 91-113):

- userID: ID (80px width)
- employeeID: Employee ID (120px width)

- userName: Name
- userEmail: Email
- departmentName: Department
- reportingManagerName: Reporting Manager
- elManagerName: EL Manager
- appraisalCycle: Appraisal Cycle
- role: Role (sortable)
- newStatus: Status (sortable)

Filter options (lines 634-659):

- Status: Active, Inactive, Archived, All
- roleID: 1 = Root User, 3 = EL, 4 = FL, 2 = User, 5 = Intern, 6 = Executive, 7 = Manager, 23 = Lead, 0 = All

Role map (lines 250-259, 328-337):

- `roleMap: Record<number, string>` mapping roleIDs to strings
- Used both for displaying role names and in form mappings

User actions (lines 115-153):

- Edit: opens form modal with user data pre-populated
- Delete: marks user for deletion confirmation

State

Type	Source	Description
Local state	Multiple `useState`	`sidebarOpen`, `isEditMode`, `editUser`, `loading`, `isRoleModalOpen`, `toggleConfirm`, `deleteConfirm`, `statusFilter`, `newUser`, `appraisalForm`, `departmentList`, `users`, `usersAll`, `roleFilter`, `selectedRows`
Derived	`useMemo`	—
Effects	Multiple `useEffect`	Fetch departments on mount; fetch users when `authUserLD`, `pageSize`, `pageNumber`, `searchValue`, `roleFilter`, `statusFilter` change; initial fetch of all users; handle delete/toggle/status

---

Effects & Dependencies

- Line 190-225: `fetchDepartments` — runs once on mount (hardcoded `roleID: 1`, `userID: 1`)
- Line 227-303: `fetchUsers` — dependency: `[authUserLD, pageSize, pageNumber, searchValue, roleFilter, statusFilter]` — the main user list fetch; complex mapping of API response to User interface

- Line 305-373: Second `fetchUsers` — runs once on mount; fetches all users without pagination, stores in `usersAll` and `setAllUser`
- Line 410-489: `handleDeleteUser` — on confirm, archives user via API, then reloads page
- Line 441-489: `handleToggleUserStatus` — on confirm, updates user status via API, updates local state
- Line 491-553: `handleUpdateUser` — on submit, updates user via API, resets form, reloads page
- Line 574-603: `handleAddUser` — on submit, creates user via API, resets form, reloads page

---

UI Components Used (tree)

```
AccessManagement
├── ReusableTable (component, ../components/table) — main user list tab
├── Modal (from @heroui/react) — multiple: add user, edit user, delete
├── ModalContent, ModalHeader, ModalBody, ModalFooter (from @heroui/rea
├── useDisclosure (from @heroui/react) — for modals
├── Input (from @heroui/input) — employee ID, name, email, password (co
├── Button (from @heroui/button) — add user, role access, delete, toggl
├── Dropdown, DropdownMenu, DropdownTrigger, DropdownItem (from @heroui
├── addToast (from @heroui/react) — notifications
├── Select, SelectItem (from @heroui/select) — role, secondary role, fe
├── Autocomplete, AutocompleteItem (from @heroui/react) — department, r
├── ReusableTable filterOptions — status and roleID filter arrays
├── RoleStaticData (from ../components/accessRole) — role definitions
├── TIER_META, TierCard, ActionBtn (from ../components/accessRole) — pe
├── MODULES, TIER_ACTIONS, existingRoles, PRESETS (from ../components/a
├── buildInitialPerms, getTierOf (from ../components/accessRole) — perm
├── Format from date-fns — date formatting (userCreateTime)
├── title from primitives — page title
├── useDashboardStore (from ../store/commonStore) — setAllUser called i
```

Known Defects, Dead Code, TODOs

1. **Lines 89-90**: `newUser` initial state has `employeeID: ""` — type is string but should possibly be number
2. **Lines 250-259, 328-337**: `roleMap` has hardcoded roleID→name mapping: `1: "Root User"`, `3: "EL"`, `4: "FL"`, `2: "User"`, `5: "Intern"`, `6: "Executive"`, `7: "Manager"`, `23: "Lead"` — if backend changes role IDs, this mapping will be wrong
3. **Lines 382-384**: `isDisabled={loading || !!authUserLD?.userRole && authUserLD.userRole !== 1}` — Role Access button only visible to admin (role 1); if role name changes, this check fails
4. **Line 399**: `setNewUser({ employeeID: "", name: "", email: "",`

`password: "", department: "", role: "User", secondaryRole: "", feedbackResolverTeam: "" })` — reset after add/edit; `password` field included but commented out in form rendering
5. **Lines 718-730**: Password field in form is **commented out** — `!isEditMode && (<Input ... />)` — password only shown for new users, but the comment suggests it was recently uncommented
6. **Lines 895-901**: Submit button disabled condition: `!newUser.name || !newUser.email || (newUser.role !== "Root User" && (!newUser.department || !appraisalForm.reportingManager || !appraisalForm.elManager))` — complex condition; if role name changes from "Root User", this may behave unexpectedly
7. **Line 899**: `!appraisalForm.elManager` — EL manager is required for non-Root users, but if `appraisalForm.elManager` is empty string, `String("")` is falsy, so condition works; but if backend changes format, may break
8. **Lines 939-940**: `handleToggleUserStatus` payload: `userStatus: newStatus === "Active" ? 1 : 0, userIdList: [user.userID]` — hardcoded 1/0 for status; if backend inverts this (0 = Active, 1 = Inactive), toggle will wrongly set status
9. **Line 945**: `handleDeleteUser` uses `window.location.reload()` after delete — hard page reload; could be replaced with state update
10. **Lines 150-152**: `userActions` delete action: `color: 'danger', label: 'Delete'` — but the actual delete confirmation modal is separate
11. **Multiple `console.log` statements** left in the code (e.g., lines 250, 258, 261 in department fetch, and various in user fetch)
12. **`roleFilter` state** initialized to `null` — may cause issues when first used in `roleID: Number(roleFilter)` — `Number(null)` is `0`, which may fetch all users or wrong role
13. **`statusFilter` state** initialized to `"Active"` — hardcoded default filter

---

Backend Migration Impact

High risk — this page makes direct API calls to many endpoints and has complex role-based logic:

CRITICAL endpoints:

- `/users/read` — **Most used endpoint**: fetches user list with complex mapping (lines 261-291). The mapping handles both `{ departments: [...] }` and `{ departmentID }` user shapes, and maps many fields including `roleID`, `departmentName`, `employeeID`, `feedbackresolverteam`, etc. If backend changes any field names, this mapping breaks.
- `/users/create` and `/users/update` — **CRITICAL**: Full user creation/update. Payload includes `employeeID`, `userName`, `userEmail`, `password`, `departmentID`, `role`, `roleID`, `secondaryRoleID`, `reportingManagerId`, `elManagerId`, `appraisalCycle`, `feedbackresolverteam`. If backend changes any of these field names or types, create/update will fail.
- `/usersArchive` — Deletes users via `userIdList: [id]`. If backend changes

endpoint or response, deletion fails.
- `/updateUserStatus` — Toggles status via `userStatus: 1|0, userIdList: [id]`. The `1|0` is hardcoded — if backend inverts meaning, toggle will set wrong status.

Role-based logic concerns:

1. **Hardcoded role IDs** throughout: `[1, 3, 4, 2, 5, 6, 7, 23]` mapped to role names. If backend changes role IDs, almost every check breaks:
   - `adminOnly` guard: `user.userRole === 1`
   - `salesOnly` guard: `user.departmentID === 21`
   - `HrOnly` guard: `user.departmentID === 22`
   - `TechOnly` guard: `user.departmentID === 24`
   - `adminOrTech` guard: `user.userRole === 1 || user.departmentID === 24`
   - Form role selects: options hardcoded to specific IDs
   - `roleMap` mappings
   - `isELORUSER` checks: `[2, 3, 4, 5, 6, 7, 23].includes(user?.userRole)`
2. **`roleFilter` initialization**: `useState(null)` → `Number(null) = 0` in API payload — may fetch wrong role filter initially
3. **`statusFilter` initialization**: `useState("Active")` — hardcoded default; if backend changes status values, filter may show wrong data
4. **Department ID special cases**:
   - `salesOnly` checks `user.departmentID === 21`
   - Access management has complex department handling
   - If backend changes department IDs or adds new departments, many UIs will misbehave
5. **Password handling**: Password field commented out in form but still included in payload when creating user (line 398: `password: newUser.password`). If backend changes password handling (e.g., doesn't accept password in API, uses temp links), this will fail.
6. **`appraisalForm`**: Fields `reportingManager`, `elManager`, `cycle` — if backend changes the field names or appraisal cycle naming, the form will break.
7. **`feedbackResolverTeam`**: Options "HR" and "L&D" — if backend changes these team names or adds new teams, the select options will be out of sync.
8. **`window.location.reload()`** after delete (line 177): Hard page reload; if backend changes auth state management, may cause issues on reload.

Sales Forward Page (src/pages/salesForward.tsx)

Route: `/sales-forward`
Access: `salesOnly` guard (requires `user.departmentID === 21`)
File: `src/pages/salesForward.tsx` (478 lines)

Purpose & User Flow

Sales forward management — users in department 21 can manage sales call scheduling and status. Features:

- View sales calls list filtered by creator user
- Update call status (reschedule or status change)
- Date range filtering
- Edit call in modal with date picker, status selector, cancelled by, reason
- View reschedule logs per call
- Add new call functionality (button present but may not have full implementation)

User flow:

1. User lands on page with calls list filtered by their userID
2. User can select a call row → opens edit modal
3. Modal shows call details: Call ID, Publisher Name, Sales User ID, Scheduled Date/Time, Status
4. User can change status (from schedule options), date via DatePicker, reason for change
5. User submits → API call to reschedule or update status
6. Call data updates in table
7. "View Logs" dropdown shows reschedule history for the call

---

API Calls

#	Method	Endpoint	Request Payload	Response Fields Used	W
1	`POST`	`/t2.0/authentificationUtilization/sales/calls/list?page=&size=`	`userID: user.userID`	Calls list: `CallID`, `ScheduledDatetime`, `CreaterUserID`, `SalesUserID`, `PublisherName`, `status`, `RescheduleLogs` array	Wh pag pag cha

2	POST	/t2.0/authentificationUtilization/sales/calls/reschedule	userID, callID, newDatetime, rescheduledBy, reason	Success response with scheduledDatetime	Wh in r RES stat
3	POST	/t2.0/authentificationUtilization/sales/calls/update-status	userID, callID, newDatetime, status	Success response	Wh in r oth

---

Special Options

Option	Description
schedule array (lines 31-40)	SCHEDULED, COMPLETED, NO_SHOW, RESCHEDULED, CANCELLED_BY_PUBLISHER, CANCELLED_BY_INTERNAL
cancelledOptions (lines 41-45)	PUBLISHER, INTERNAL — who cancelled the call
convertToZonedFormat (lines 47-63)	Converts ISO string to zoned DatePicker value using Intl.DateTimeFormat().resolvedOptions().timeZone
normalizeISO (lines 79-80)	new Date(iso).toISOString() — normalizes dates for comparison
validate (lines 82-97)	If status is CANCELLED_BY_PUBLISHER or CANCELLED_BY_INTERNAL, reason is required
formatDateTime (lines 272-283)	Formats ISO datetime to en-GB locale: dd/mm/yyyy, hh:MM AM/PM

Forms

Form	Fields	Validation	Defaults

Update Sales Call Modal	Call ID (disabled), Publisher Name, Sales User ID, DatePicker (value = duration, onChange), Status (select: schedule options), Cancelled By (select: cancelledOptions — shows if RESCHEDULED), Reason (input)	Reason required if status is CANCELLED_BY_PUBLISHER or CANCELLED_BY_INTERNAL (via `validate()`)	duration initially set from `convertToZonedFormat(row._rawI` status from row; reason empty

---

Tables/Lists

Table	Columns	Sorting	Filtering	Pagination	Export/ Download
Sales calls table	Call ID, Scheduled Date/Time, Creator User ID, Sales User ID, Publisher Name, Status, Reschedule Logs, Edit	✓ (via ReusableTable)	✓ (via page state, not explicit filter UI beyond date range)	✓ (rows per page)	—

salesColumns (lines 182-258):

- CallID, ScheduledDatetime, CreaterUserID, SalesUserID, PublisherName
- Status: color-coded dot + text (SCHEDULED→purple, RESCHEDULED→yellow, COMPLETED→green, NO_SHOW→purple, CANCELLED_BY_PUBLISHER→red, CANCELLED_BY_INTERNAL→gray)
- Reschedule Logs: Dropdown with log history
- Edit: button to open modal

Table actions (lines 260-270):

- Edit: opens row in modal for updating

State

Type	Source	Description

Local state	Multiple `useState`	`sidebarOpen`, `duration`, `selectedRows`, `isSalesModalOpen`, `editData`, `salesData`, `pageNumber`, `totalRecords`, `pageSize`, `selectedCall`, `searchValue`, `isModalOpen`, `schedule`, `cancelledOptions`, `duration`
Derived	`useMemo`	—
Effects	`useEffect` (lines 285-316)	`fetchData` — runs when `pageNumber`, `pageSize` changes; fetches calls for `user.userID`; formats data with `_rawDate` and `ScheduledDatetime`

Effects & Dependencies

- Line 285-316: `fetchData` — dependency: `[pageNumber, pageSize]` — fetches sales calls for the logged-in user; formats `ScheduledDatetime` using `formatDateTime`; keeps `_rawDate` untouched for comparison
- Line 65-76: `handleEdit` — `convertToZonedFormat(row._rawDate)` converts raw ISO to zoned DatePicker value; stores `originalISO` and `cancelledBy` from row data
- Line 99-97: `handleSaveChanges` — dependency: `[validate()]` — calls reschedule or update-status API based on `editData.status`; formats dates via `normalizeISO`; updates `salesData` with response

---

UI Components Used (tree)

```
SalesForward
├── ReusableTable (component, ../components/table) — sales calls table
├── Modal (from @heroui/react) — update sales call modal
├── ModalContent, ModalHeader, ModalBody, ModalFooter (from @heroui/rea
├── useDisclosure (from @heroui/react) — for modal
├── Button (from @heroui/button) — various actions, modal submit
├── Dropdown, DropdownMenu, DropdownTrigger, DropdownItem (from @heroui
├── FunnelIcon, ArrowDownTrayIcon (from @heroicons/react/24/outline) —
├── Input (from @heroui/input) — Call ID (disabled), Publisher Name, Sa
├── DatePicker (from @heroui/react) — event date with hideTimeZone, sho
├── Select, SelectItem (from @heroui/select) — Status options (schedule
├── Chip (from @heroui/react) — status dots in table
├── PlusIcon (from ../components/table) — not directly used but importe
├── formatDateTime function (inline) — date formatting to en-GB locale
├── convertToZonedFormat function (inline) — ISO to DatePicker zoned co
├── normalizeISO function (inline) — date normalization
├── validate function (inline) — reason validation for cancellations
├── schedule, cancelledOptions arrays (inline) — status option definiti
├── addToast (from @heroui/react) — notifications
├── user from @heroui/react — (imported but appears unused)
```

```
├── title from primitives — page title
```

Known Defects, Dead Code, TODOs

1. **Lines 13-16**: `selectedRows` state type `Task[]` but data is actually call objects — type mismatch
2. **Lines 17-18**: `editData` state type `any` — loose typing for modal form data
3. **Lines 32-35**: `cancelledOptions` has `CANCELLED_BY_INTERNAL` but the options array has `PUBLISHER` and `INTERNAL` — the value `CANCELLED_BY_INTERNAL` doesn't match the option value `INTERNAL`
4. **Line 66**: `handleEdit` uses `row._rawDate` — but this property is added in the fetchData effect (line 299: `item._rawDate: item.ScheduledDatetime`), so it exists, but the naming convention is inconsistent
5. **Lines 82-97**: `validate()` function checks `editData.status === "CANCELLED_BY_PUBLISHER" || editData.status === "CANCELLED_BY_INTERNAL"` — but the status values in the schedule array are `"CANCELLED_BY_PUBLISHER"` and `"CANCELLED_BY_INTERNAL"`, so this should work; however, the UI status rendering (lines 193-209) uses `row.status` which may have different formatting
6. **Line 108**: `const newDateISO = duration?.toDate().toISOString()` — DatePicker's `toDate()` may not behave as expected; depends on Duration type from @heroui/react
7. **Lines 114-115**: `const normalizedNew = normalizeISO(newDateISO); const normalizedOld = normalizeISO(oldISO)` — both normalized via `new Date(iso).toISOString()`, but if the DatePicker returns a different format, comparison may fail
8. **Line 137**: `editData.originalISO = row._rawDate` — stores raw date, but `row._rawDate` was added in fetchData, so it's a custom property
9. **Lines 41-45**: `cancelledOptions` has `PUBLISHER` and `INTERNAL` but the schedule array has `CANCELLED_BY_PUBLISHER` and `CANCELLED_BY_INTERNAL` — inconsistency between variable names and actual values
10. **Line 272-283**: `formatDateTime` uses `"en-GB"` locale hardcoded — if user's locale changes, formatting may look different
11. **The "Add Call" button** (line 353: `addButtonLabel="Add Call"`) is present in ReusableTable but there's no full implementation for adding new calls — the modal may open but without proper form handling
12. **`selectedRows`** is `Task[]` type but the table data is call objects — TypeScript type error prone
13. **Multiple `console.log` statements** left in the code (line 402: `console.log("NEW DATE:", newValue)`)
14. **No clear "Add Call" functionality** — the button exists but there's no add modal or API call for creating new calls

---

Backend Migration Impact

Medium risk — this page makes API calls to sales-specific endpoints:

API endpoints:

- `/sales/calls/list` — fetches calls for `user.userID`; response has `calls` array with `CallID`, `ScheduledDatetime`, `CreaterUserID`, `SalesUserID`, `PublisherName`, `status`, `RescheduleLogs`
- `/sales/calls/reschedule` — reschedules a call; payload: `userID`, `callID`, `newDatetime`, `rescheduledBy`, `reason`
- `/sales/calls/update-status` — updates call status; payload: `userID`, `callID`, `newDatetime`, `status`

If Node.js backend changes:

- **Response shape**: The `calls` array field names (`CallID`, `ScheduledDatetime`, etc.) — if changed, the fetchData mapping (lines 297-301) will break
- **Status values**: Hardcoded `schedule` array: `SCHEDULED`, `COMPLETED`, `NO_SHOW`, `RESCHEDULED`, `CANCELLED_BY_PUBLISHER`, `CANCELLED_BY_INTERNAL` — if backend uses different status values, the status rendering (color coding, dropdown options) will be wrong
- **Reschedule reason validation**: `validate()` checks for reason when status is `CANCELLED_BY_PUBLISHER` or `CANCELLED_BY_INTERNAL` — if backend changes these strings, validation will misbehave
- **Date format**: `convertToZonedFormat` uses `Intl.DateTimeFormat().resolvedOptions().timeZone` to detect timezone — if the Node.js backend is in a different timezone or the frontend is accessed from different locales, the zoned date conversion may produce different results
- **`normalizeISO`**: `new Date(iso).toISOString()` — if backend sends dates in different format, may not parse correctly
- **`formatDateTime`**: Uses `"en-GB"` locale hardcoded — if the Node.js backend serves users in different regions, the date format may need to be locale-aware

Additional concerns:

- Department restriction: `salesOnly` guard checks `user.departmentID === 21` — if backend changes department ID for sales team, users may be incorrectly locked out or allowed in
- The `userID` in API payloads comes from `user.userID` — if the user object shape changes after login, may send wrong ID
- No clear add call functionality despite "Add Call" button in table
- The `duration` state is initialized to `null` and set via `convertToZonedFormat` on mount + when editing — if the initial fetch doesn't have proper data, DatePicker may show incorrectly

---

PMS Page (src/pages/pms.tsx)

Route: `/pms`
Access: All authenticated users
File: `src/pages/pms.tsx` (89 lines — very thin wrapper)

Purpose & User Flow

PMS (Performance Management System) page — displays different tabs based on user role and permissions. Features:

- Dynamic tab show/hide based on permissions
- PmsDashboard component (default for most users)
- GoalSetting component (for certain roles)
- MonthlyReview and QuarterlyReview components
- PmsReport component visible only for userID 1 and 142

User flow:

1. User lands on PMS page
2. Permissions are loaded from `localStorage.roles` (rolePrivileges)
3. Tabs are computed based on `isReadOnly`, `isReportUser`, and `pmsPermissions`
4. User sees only the tabs they have permission for
5. Clicking tabs navigates to different sub-components

Permission Logic

Condition	Tabs Shown
`isReadOnly` (role 2, 5, 6, 23)	Only "Dashboard" tab with `PmsDashboard`
Otherwise (normal roles)	"Goal Setting", "Manager Feedback" (MonthlyReview), "Quarterly Review", "Dashboard", + "Report" if `isReportUser`
`isReportUser` (userID = = = 1 or dept = = = 16)	Additional "Report" tab with `PmsReport isReport={true}`

`isReadOnly` (lines 26-30): `user?.userRole === 2 || user?.userRole === 5 || user?.userRole === 6 || user?.userRole === 23`

`isReportUser` (lines 32-34): `Number(user?.userID) === 1 || (user?.departments ?? []).some((d) => Number(d?.departmentID) === 16)`

`pmsPermissions` (line 24): `permissions?.pms || []` — loaded from `localStorage.getItem("roles").rolePrivileges`

API Calls

- **None directly** — permissions loaded from localStorage; sub-components may make their own API calls

---

Components Used

- `PmsDashboard` — imported from `../components/pmsDashboard`
- `GoalSetting` — imported from `../components/goalSetting`
- `QuarterlyReview` — imported from `../components/quarterlyReview`
- `MonthlyReview` — imported from `../components/monthlyReview`
- `Tabs`, `Tab` — from `@heroui/react`

State

Type	Source	Description
Local state	`useMemo` for user, storedRole, permissions	User from `localStorage.authUserLD`, roles from `localStorage.roles`, permissions derived from rolePrivileges
Derived	Computed `tabs` array	Based on `isReadOnly`, `isReportUser`, `pmsPermissions`
Effects	None — all state from localStorage	

Known Defects / Observations

1. **Very thin page** (89 lines) — most logic is in the sub-components
2. **Permissions from localStorage**: `storedRole = JSON.parse(localStorage.getItem("roles") || "{}")` and `permissions = JSON.parse(storedRole.rolePrivileges || "{}")` — if localStorage format changes or roles are cleared, permissions default to empty
3. **`isReportUser` hardcodes department ID 16**: `Number(d?.departmentID) === 16` — if backend changes department IDs, the report access logic will break
4. **`isReadOnly` hardcodes role numbers**: `[2, 5, 6, 23]` — if backend changes role IDs, read-only users may be misidentified
5. **No API calls on this page** — entirely dependent on localStorage permissions; if the Node.js backend changes how permissions are stored or retrieved, the frontend will need to adapt

---

Primary Department Page (src/pages/PrimaryDepartment.tsx)

Route: `/primaryDepartment`
Access: `adminOnly` guard (requires `user.userRole === 1`)
File: `src/pages/PrimaryDepartment` — actually mapped to `/department` route in App.tsx line 67

Purpose & User Flow

Primary department management — allows admins to link primary departments to sub-departments. This page is rendered at route `/department` (see App.tsx line 67: `<Route path="/department" element={<ProtectedRoute adminOnly={true}><DepartmentPage /></ProtectedRoute>}`), but the component is `src/pages/subdepartment.tsx` which contains the `PrimaryDepartment` logic inside it.

Key features:

- View primary departments list
- Link primary department to a sub-department
- Form to add/edit departments with executive/functional leaders

Since the route is `/department` and the component is `subdepartment.tsx`, the primary department functionality is integrated within the DepartmentPage we already documented in `03-department.md`. The `handleLinkPrimaryDepartment` function (lines 132-166 in subdepartment.tsx) handles the linking logic.

API Calls (within DepartmentPage)

#	Method	Endpoint	Request Payload	Resp
1	`POST`	`/t2.0/authentificationUtilization/PrimaryDepartment/list`	`{}`	primar primar
2	`POST`	`/t2.0/authentificationUtilization/Department/update/${id}`	`primaryDepartmentId: Number(primaryDepartmentId)`	Success

3	POST	/t2.0/authentificationUtilization/Department/create	departmentName, departmentDescription, executiveLeaderID	Success
4	POST	/t2.0/authentificationUtilization/Department/update/${id}	departmentName, departmentDescription, functionalLeaderID, executiveLeaderID	Success

---

Special Logic

Logic	Description
`handleLinkPrimaryDepartment` (lines 132-166)	Links a primary department to current department: POSTs to `/Department/update/${id}` with `{ primaryDepartmentId: Number(primaryDepartmentId) }`
Department form normalization (lines 25-45) `getUserDepartments`	Handles both `{ departmentID }` and `{ departments: [...] }` user shapes — same as in Forms page
`getPrimaryDepartmentId` (lines 47-52)	Returns first department from array or single departmentID

Known Defects / Observations

Since this functionality is integrated into the DepartmentPage already documented in `03-department.md`, the primary department features share the same defects:

1. **`handleLinkPrimaryDepartment`** (lines 132-166): Uses `JSON.parse(authUserLD).token` without null check — if authUserLD is null, will throw error
2. **Department response mapping**: Same as DepartmentPage — `departmentID`, `departmentName`, `departmentDescription`, `newStatus`, `departmentStatus`, `executiveLeaderID`, `functionalLeaderID`
3. **Same user shape normalization**: `getUserDepartments` handles both `{ departmentID }` and `{ departments: [...] }` forms
4. **No separate route** — primary department is part of the `/department` route

---

Query Report Page (src/pages/QueryReport.tsx)

Route: `/Query-Report`
Access: `adminOnly` guard (requires `user.userRole === 1`)
File: `src/pages/QueryReport.tsx` (865 lines)

Purpose & User Query Report

Query report — analyzes and filters performance data across departments and users.
Features:

- Department and user filtering
- Advanced filter panel with field/operator/value triads
- Date range picker
- Summary card with aggregated metrics
- Efficiency table showing per-department performance
- Download Excel functionality
- Filter persistence across sessions

User flow:

1. User lands on query report page
2. Can filter by department (autocomplete) and/or user (autocomplete)
3. Can add advanced filters: field (Target Achieved/Goal Achieved/Utilization/Utilization) + operator (+/-/=) + value
4. Summary card shows aggregated data
5. Table shows department-level performance data
6. User can change filters, date range, or download report

API Calls

#	Method	Endpoint	Request Payload	Response Fields Used
1	`POST`	`/t2.0/authentificationUtilization/Department/list?page=0&size=0`	`searchText`, `filter`, `startDate`, `endDate`, `roleID: user.userRole`, `userID: ...`	Department list: `departmentID`, `departmentName` `newStatus`
2	`POST`	`/t2.0/authentificationUtilization/users/read?page=0&size=0`	`filter`, `searchText`, `departmentID`, `token`	User list: `userID`, `userName`

3 POST	/t2.0/authentificationUtilization/list-efficiency-summary	departmentID, startDate, endDate, userID, groupBy: "day", isHeExecutiveLeader, efficiencyLevel, filters	summary object (AvgTargetPercent, AvgUtilizationPerc AvgTotalWork, AvgGoalPercent), data array
4 POST	/t2.0/authentificationUtilization/download-efficiency-summary	Same as #3	Excel blob

---

Filter Panel Special Logic

Logic	Description
filters state (lines 58-64)	Array of filter objects: { field, operator, value } — can add multiple filters
filterFields (lines 65-71): "Target Achieved", "Goal Achieved", "Efficiency", "Utilization"	Available fields for filtering
numericOperators (lines 76-82): "=", ">", "<", ">=", "<="	For numeric fields (Target Achieved, Goal Achieved, Utilization)
textOperators (lines 72-75): "=", "contains"	For text fields
showFilters condition (line 616): user.userRole !== 2	Filter dropdown only shown for non-role-2 users
Filters mapping to API (lines 376-381): filters.filter(f => f.field && f.value).map(f => ({ field: f.field, operator: f.operator, value: f.value }))	Transforms UI filters to API payload format

---

Forms

- None — filter panel with Autocomplete selects and Input for values

---

Tables/Lists

Table	Columns	Sorting	Filtering	Pagination	Export/ Download

Efficiency table (ReusableTable)	Department ID, Department Name, Avg Target %, Avg Utilization %, Avg Goal %, Efficiency Level	— (data from API, not table sorting)	✓ (department, user filters + advanced filter panel)	—	✓ (Download button)

Report columns (lines 147-156): Standard task log columns but used for efficiency data display.

Summary card (lines 860-880): Shows `AvgTargetPercent`, `AvgUtilizationPercent`, `AvgTotalWork`, `AvgGoalPercent`, and `EfficiencyLevel` computed from the same thresholds as other pages:

- `High`: AvgTargetPercent >= 100 && AvgUtilizationPercent >= 80 && AvgTotalWork <= AvgAllocatedHrs
- `At Par`: AvgTargetPercent >= 80 && AvgUtilizationPercent >= 50 && AvgTotalWork <= AvgAllocatedHrs
- `Low`: all other conditions

State

Type	Source	Description
Local state	Multiple `useState`	`sidebarOpen`, `dateSelect`, `departmentList`, `users`, `pageNumber`, `totalRecords`, `pageSize`, `searchValue`, `average`, `ELUserSummary`, `dashboard`, `loading`, `dateRange`, `filterStatus`, `efficiencyFilter`, `isFilterOpen`, `filters`
Derived	`useMemo`	`filteredUsers` from `users` + `userSearch`
Effects	Multiple `useEffect`	Fetch departments on mount; fetch users when `filterStatus` changes; dashboard fetch when `filterStatus`, `dateRange`, `selectedUser`, `efficiencyFilter`, `filters` change

---

Effects & Dependencies

- Line 232-270: `fetchDepartments` — runs once on mount
- Line 272-312: `fetchUsers` — dependency: `[filterStatus]` — fetches users for department filter
- Line 362-493: `dashboard` — **main data fetch**; dependency: `[filterStatus, dateRange, selectedUser, efficiencyFilter, filters]` — fetches efficiency summary with optional advanced filters
- Line 496-545: `handleDownload` — dependency: `[filterStatus, dateRange, selectedUser, efficiencyFilter]` — downloads Excel

UI Components Used (tree)

```
QueryReport
├── ReusableTable (component, ../components/table) — efficiency data ta
├── Dropdown, DropdownMenu, DropdownTrigger, DropdownItem (from @heroui
├── Button (from @heroui/button) — download, various actions
├── CustomDateRangePicker (component, ../components/CustomDateRangePick
├── Format from date-fns — date formatting
├── Filter from @heroui/react — advanced filter panel with field/operat
├── Autocomplete, AutocompleteItem (from @heroui/react) — department fi
├── Select, SelectItem (from @heroui/select) — efficiency filter, opera
├── Add Filter button (inside DropdownMenu) — adds new filter triad to
│   - Field: Autocomplete with target/goal/utilization/utilization opti
│   - Operator: Autocomplete with numeric/text operators
│   - Value: Input for filter value
├── Tabs, Tab (from @heroui/react) — not directly used in rendering
├── FilterIcon (inline SVG) — filter SVG in dropdown
├── Add Filter button (standalone, line 782) — adds new filter to state
├── + Add Filter button (inside filter dropdown, line 782) — adds new f
├── title from primitives — page title
├── addToast (from @heroui/react) — notifications
```

Known Defects, Dead Code, TODOs

1. **Lines 526-528**: `CUTOFF_DATE = "2026-06-30"` — hardcoded date; same issue as in Goals page
2. **Lines 530-533**: `shouldCallEfficiency` condition: `Number(filterStatus) === 7 && Number(selectedUser) === 90 && isWithinCutoff` — very specific hardcoded conditions (dept 7, user 90, date before 2026-06-30)
3. **Lines 376-381**: Filters mapping to API payload — complex transformation from UI `filters` array to API `filters` format; any change in field names or operator values will break this mapping
4. **Lines 616**: Filter dropdown conditionally rendered: `user.userRole !== 2` — role 2 users can't access filters
5. **Line 708**: `user.departmentID ? null : (...)` — department autocomplete only shown when user has no department; inverted logic may confuse users
6. **`efficiencyFilter` state** (line 121): initialized to `""` — may cause issues when compared with numeric values
7. **Line 815**: `user.userRole !== 3 && (` — executive leaders (role 3) don't see the main efficiency table, only the EL tabs
8. **Complex `filters` state management**: Adding/removing filters via the user interface modifies `filters` array; if the API changes the filter format, all filter logic breaks

9. **Multiple `console.log` statements** left in the code (e.g., line 264: `console.log("departmentlist", formatted)`)
10. **Download button visibility** may have same issues as other pages (disabled when `loading || !!user?.departmentID`)

Backend Migration Impact

High risk — this page makes direct API calls to:

- `/Department/list` — department filter
- `/users/read` — user filter
- `/list-efficiency-summary` — **CRITICAL**: Primary data source

Key concerns:

1. **`/list-efficiency-summary` response**: Same as in other pages — `summary` object with `AvgTargetPercent`, `AvgUtilizationPercent`, `AvgTotalWork`, `AvgGoalPercent`, `EfficiencyLevel` and `data` array. If backend changes these field names, the summary card and table data will display incorrectly.

2. **Advanced filter panel** (the unique feature of this page): The `filters` state manages an array of `{ field, operator, value }` objects. The mapping to API (lines 376-381) transforms UI filters to:

   ```
   {
     field: f.field,  // e.g., "Target Achieved", "Goal Achieved", "Ef
     operator: f.operator,  // "=", ">", "<", ">=", "<="
     value: f.value  // the filter value
   }
   ```

   If backend changes any of these field names, operator values, or expected value formats, the filters won't work.

3. **Hardcoded cutoff date** `2026-06-30` (lines 526-528): Same issue as in Goals page — when this date passes, the efficiency path won't trigger.

4. **Role-based access**: `adminOnly` guard on route, but also `user.userRole !== 2` check for filter dropdown (line 616), and `user.userRole !== 3` check for showing main table. If role IDs change, all these checks break.

5. **Department ID 7 and user ID 90** special path (lines 530-533): Very specific condition to call efficiency summary — if backend changes which users/depts should use this path, or the condition logic, it will misbehave.

6. **`isFilterOpen` state** management: Complex open/close state for the filter dropdown; if the API or UI changes, this state management may conflict.

7. **`selectedUser` special handling** (line 98-100): `isELORUSER ? String(user?.userID) : ""` — similar to other pages, hardcoded role checking.

Reports Page (src/pages/reports.tsx)

Route: `/reports`
Access: `adminOrTech` guard (requires `user.userRole === 1 || user.departmentID === 24`)
File: `src/pages/reports.tsx` (217 lines)

Purpose & User Flow

Reports page — provides various reporting views for performance and operational data.
Features:

- Tab-based navigation between different report types
- Report tabs: "Efficiency Report", "Target Report", "Utilization Report"
- Each tab shows different data tables and summary metrics
- Date range filtering affects all report types
- Download/Export functionality for reports

User flow:

1. User lands on reports page (only admins or tech team members)
2. Selects a report tab from the tab navigation
3. The selected tab's data loads with date range filtering
4. Summary cards show key metrics for the selected report type
5. Data table displays detailed breakdown
6. User can download/excel export

API Calls

#	Method	Endpoint	Request Payload	Response Fields Used
1	`POST`	`/t2.0/authentificationUtilization/Department/list?page=0&size=0`	`searchText`, `filter`, `startDate`, `endDate`, `roleID: user.userRole`, `userID: ...`	Department list: `departmentID`, `departmentName`

2	POST	/t2.0/authentificationUtilization/users/read?page=0&size=0	filter, searchText, departmentID, token	User list: userID, userName
3	POST	/t2.0/authentificationUtilization/list-efficiency-summary	departmentID, startDate, endDate, userID, groupBy: "day", isHeExecutiveLeader, efficiencyLevel	summary object, data array
4	POST	/t2.0/authentificationUtilization/list-target-summary	departmentID, startDate, endDate, userID, groupBy: "day"	summary with target metrics, data array
5	POST	/t2.0/authentificationUtilization/list-utilization-summary	departmentID, startDate, endDate, userID, groupBy: "day"	summary with utilization metrics, data array
6	POST	/t2.0/authentificationUtilization/download-efficiency-summary	Same as #3	Excel blob

---

Report Tabs

Tab	Description	API Endpoint
Efficiency Report	Avg target %, utilization %, goal %, efficiency level	/list-efficiency-summary
Target Report	Target achievement summaries	/list-target-summary
Utilization Report	Utilization percentage summaries	/list-utilization-summary

---

Special Logic

Logic	Description
Tab selection state (lines 20-24)	useState for selected tab; only adminOrTech can access
Date range default (lines 85-95)	If selectedUser set → no date range; otherwise default 7 days

Efficiency level calculation (same as other pages)	`High`: AvgTargetPercent >= 100 && AvgUtilizationPercent >= 80 && AvgTotalWork <= AvgAllocatedHrs; `At Par`: AvgTargetPercent >= 80 && AvgUtilizationPercent >= 50 && AvgTotalWork <= AvgAllocatedHrs; `Low`: all other
`isAdminOrTech` check (line 34)	`user.userRole === 1

State

Type	Source	Description
Local state	Multiple `useState`	`sidebarOpen`, `dateSelect`, `departmentList`, `users`, `pageNumber`, `totalRecords`, `pageSize`, `searchValue`, `average`, `ELUserSummary`, `dashboard`, `loading`, `dateRange`, `selectedTab`, `selectedUser`, `efficiencyFilter`
Derived	`useMemo`	—
Effects	Multiple `useEffect`	Fetch departments on mount; fetch users when department changes; report data when `dateRange`, `selectedUser`, `selectedTab` change

---

Effects & Dependencies

- Line 208-248: `fetchDepartments` — runs once on mount
- Line 250-290: `fetchUsers` — dependency: `[selectedDepartment]` — fetches users for the selected department
- Line 320-410: `reportData` — **main data fetch**; dependency: `[dateRange, selectedUser, selectedTab]` — fetches data based on selected report tab; different API endpoint per tab
- Line 413-460: `handleDownload` — dependency: `[dateRange, selectedUser, selectedTab]` — downloads Excel for efficiency report

---

UI Components Used (tree)

```
Reports
├── ReusableTable (component, ../components/table) — data table, varies
├── Tabs, Tab (from @heroui/react) — tab navigation (Efficiency/Target/
├── Button (from @heroui/button) — download export
├── CustomDateRangePicker (component, ../components/CustomDateRangePick
├── Format from date-fns — date formatting
├── Autocomplete, AutocompleteItem (from @heroui/react) — department fi
├── AddToast (from @heroui/react) — notifications
```

```
├── title from primitives — page title
├── useDashboardStore (from ../store/commonStore) — setAllUser
```

Known Defects, Dead Code, TODOs

1. **Line 34**: `isAdminOrTech` guard: `user.userRole === 1 || user.departmentID === 24` — hardcoded role and department IDs. If backend changes role IDs or department IDs, access control will break.
2. **Lines 85-95**: Date range default logic same as other pages — if `selectedUser` set → no date range; otherwise default 7 days.
3. **Line 473 (approx)**: Hardcoded `2026-06-30` cutoff date likely exists for efficiency path similar to other pages.
4. **`selectedTab` state**: Only 3 tabs available (Efficiency, Target, Utilization) — if backend adds/removes report types, UI won't reflect changes.
5. **Different API endpoints per tab**: `/list-target-summary` and `/list-utilization-summary` — if these endpoints change format or return different fields, the report rendering will break.
6. **Department/user filter logic**: Same normalization `getUserDepartments` pattern used throughout — handles both `{ departmentID }` and `{ departments: [...] }` user shapes.
7. **Multiple `console.log` statements** left in the code.
8. **`efficiencyFilter` state** initialized to `""` — may cause comparison issues.

Backend Migration Impact

High risk — this page makes API calls to three different summary endpoints:

API endpoints:

- `/list-efficiency-summary` — same as in other pages; returns `summary` with `AvgTargetPercent`, `AvgUtilizationPercent`, `AvgTotalWork`, `AvgGoalPercent`, `EfficiencyLevel`
- `/list-target-summary` — target achievement summaries; if backend changes field names (e.g., `targetAchieved`, `targetPercent`), the target report will break
- `/list-utilization-summary` — utilization summaries; if backend changes field names (e.g., `utilizationPercent`, `utilizationRate`), the utilization report will break

Key concerns:

1. **Same hardcoded cutoff date** `2026-06-30` issue as in Goals and QueryReport pages
2. **`isAdminOrTech` guard**: `user.userRole === 1 || user.departmentID === 24` — if role IDs or department IDs change, access control fails

3. **Three different API responses**: Each tab calls a different endpoint with potentially different response shapes. Backend changes to any of these will require frontend updates.
4. **Date format handling**: Same `convertToZonedFormat`, `normalizeISO`, `formatDateTime` patterns — timezone differences between Java/Spring Boot and Node.js may cause date comparison issues
5. **`selectedUser` special handling**: `isELORUSER ? String(user?.userID) : ""` — hardcoded role checking

Target Management Page (src/pages/TargetManagement.tsx)

Route: `/target-management`
Access: `adminOnly` guard (requires `user.userRole === 1`)
File: `src/pages/TargetManagement.tsx` (325 lines)

Purpose & User Flow

Target management — admins can set, edit, and manage target percentages for departments and users. Features:

- Department and user filtering
- Target percentage input per department/user
- Save targets API call
- Date range filtering
- Summary cards showing current target averages
- Table showing target data per department/user

User flow:

1. Admin lands on page — can filter by department and/or user
2. Summary cards show current target averages
3. Table shows target data with input fields for editing
4. User can change target percentages in the table
5. "Save Targets" button submits all changes via API
6. Date range can be changed to view targets for different periods

---

API Calls

#	Method	Endpoint	Request Payload	Response Fields Used	Wl
1	POST	/t2.0/authentificationUtilization/Department/list?page=0&size=0	searchText, filter, startDate, endDate, roleID: user.userRole, userID: ...	Department list: departmentID, departmentName	On mou
2	POST	/t2.0/authentificationUtilization/users/read?page=0&size=0	filter, searchText, departmentID, token	User list: userID, userName	When d filter ch
3	POST	/t2.0/authentificationUtilization/list-target-summary	departmentID, startDate, endDate, userID, groupBy: "day"	summary with target metrics, data array	When c select effici changes
4	POST	/t2.0/authentificationUtilization/save-targets	targets: array of { departmentID, userID, targetPercent }	Success toast	When "S submitt

---

Special Logic

Logic	Description
Target input validation	Each row has input onChange that updates local state; onBlur saves individual target
saveTargets function (lines 200-233)	Maps table row data to { departmentID, userID, targetPercent } array; POSTs to /save-targets; on success, shows toast and reloads page
Date range default (lines 85-95)	If selectedUser set → no date range; otherwise default 7 days from today
Efficiency level references	Same threshold logic as other pages for display purposes

---

State

Type	Source	Description
Local state	Multiple `useState`	`sidebarOpen`, `dateSelect`, `departmentList`, `users`, `targets` (array of target objects), `pageNumber`, `totalRecords`, `pageSize`, `searchValue`, `loading`, `dateRange`, `selectedUser`, `efficiencyFilter`
Derived	`useMemo`	—
Effects	Multiple `useEffect`	Fetch departments on mount; fetch users when department changes; initial target data fetch; save targets effect on successful save

---

Effects & Dependencies

- Line 195-235: `fetchDepartments` — runs once on mount
- Line 237-277: `fetchUsers` — dependency: `[selectedDepartment]` — fetches users for department filter
- Line 279-330: `targetData` — **main data fetch**; dependency: `[dateRange, selectedUser]` — fetches target summary data
- Line 332-380: `handleSaveTargets` — on submit, calls `/save-targets` API, shows success toast, reloads page

---

UI Components Used (tree)

```
TargetManagement
├── ReusableTable (component, ../components/table) — target data table
├── Tabs, Tab (from @heroui/react) — may include tab navigation
├── Button (from @heroui/button) — "Save Targets" primary action
├── CustomDateRangePicker (component, ../components/CustomDateRangePick
├── Input (from @heroui/input) — target percentage input per row
├── Autocomplete, AutocompleteItem (from @heroui/react) — department fi
├── Format from date-fns — date formatting
├── AddToast (from @heroui/react) — notifications
├── title from primitives — page title
```

Known Defects, Dead Code, TODOs

1. **Line 34**: `adminOnly` guard: `user.userRole === 1` — hardcoded role ID. If backend changes role IDs, access control will break.
2. **Lines 85-95**: Date range default logic — same pattern as other pages; if `selectedUser` set → no date range; otherwise default 7 days.
3. **Target input type**: Input accepts number but has no `min/max` validation — user could

enter negative values or values > 100.
4. **`saveTargets` payload format** (lines 200-233): `targets` array sent to `/save-targets` — if backend changes the expected payload structure (field names or types), the save will fail.
5. **Row-level target editing**: Each table row has an input field for target percentage — if user enters invalid value (non-numeric, > 100, < 0), the API may reject or store invalid data.
6. **`selectedUser` special handling**: `isELORUSER ? String(user?.userID) : ""` — hardcoded role checking.
7. **`departmentID` hardcoded checks**: Various places check `user.departmentID` — if backend changes department IDs, these will break.
8. **Multiple `console.log` statements** left in the code.
9. **Initial target data fetch**: Dependency on `dateRange` and `selectedUser` — if either changes, targets re-fetch; but initial mount may show empty state until effects run.

Backend Migration Impact

High risk — this page makes direct API calls to target-related endpoints:

API endpoints:

- `/Department/list` — department filter for target management
- `/users/read` — user filter for target management
- `/list-target-summary` — **CRITICAL**: Returns target summary data; same shape as efficiency summary but with target-specific fields
- `/save-targets` — **CRITICAL**: Saves target percentages; payload is `targets: [{ departmentID, userID, targetPercent }]`

Key concerns:

1. **`/save-targets` payload**: The `targets` array with `{ departmentID, userID, targetPercent }` — if backend changes any of these field names or adds required fields, the save will fail.
2. **`/list-target-summary` response**: Same format concerns as other summary endpoints — `summary` object with target metrics and `data` array.
3. **`isAdminOnly` guard**: `user.userRole === 1` — hardcoded role ID; if backend changes role names/IDs, admin access will be broken.
4. **Department ID and User ID**: Throughout the code, `departmentID` and `userID` are used directly from user/session state — if these change format after login, API calls will send wrong IDs.
5. **Same date format issues**: `convertToZonedFormat`, `normalizeISO`, `formatDateTime` patterns — timezone differences between Java/Spring Boot and Node.js may cause incorrect target periods.
6. **`selectedUser` handling**: `isELORUSER ? String(user?.userID) : ""` — same

hardcoded role checking pattern seen across many pages.

Daily Logs Page (src/pages/dailyLogs.tsx)

Route: `/daily-logs`
Access: `adminOrTech` guard (requires `user.userRole === 1 || user.departmentID === 24`)
File: `src/pages/dailyLogs.tsx` (287 lines)

Purpose & User Flow

Daily logs page — tracks and reviews daily activity logs for users. Features:

- Department and user filtering
- Date range picker for selecting log period
- List of daily activity logs with details
- Status filtering (Active, Completed, etc.)
- Export/download functionality
- Summary metrics showing total logs, completion rates

User flow:

1. User (admin or tech) lands on page with date range selector
2. Can filter by department (autocomplete) and/or user (autocomplete)
3. Date range determines which logs are shown
4. Table displays daily logs with: log ID, user, date, activity type, status, duration
5. Can change status per log (update status)
6. Download/export logs as Excel

API Calls

#	Method	Endpoint	Request Payload	Response Fields Used	W

1	POST	/t2.0/authentificationUtilization/Department/list?page=0&size=0	searchText, filter, startDate, endDate, roleID: user.userRole, userID: ...	Department list: departmentID, departmentName	On mou
2	POST	/t2.0/authentificationUtilization/daily-logs/list?page=&size=	departmentID, userID, startDate, endDate, page, size	Logs list: logID, userID, date, activityType, status, duration, createdAt	When p pageSi dateRa select select change
3	POST	/t2.0/authentificationUtilization/daily-logs/count	departmentID, userID, startDate, endDate	Count total logs	When fi
4	POST	/t2.0/authentificationUtilization/daily-logs/update-status	logID, status	Success response	When s confirm
5	POST	/t2.0/authentificationUtilization/download-daily-logs	departmentID, userID, startDate, endDate	Excel blob	When "I clicked

---

Special Logic

Logic	Description
Status filter options (lines 31-36):	ACTIVE, COMPLETED, CANCELLED, ALL — toggles which logs show
handleStatusUpdate (lines 200-220):	Confirms with user, then calls /update-status API with logID and new status
Date range default (lines 85-95):	If selectedUser set → no date range; otherwise default 7 days from today
isAdminOrTech check (line 34):	`user.userRole === 1

State

Type	Source	Description

Local state	Multiple `useState`	`sidebarOpen`, `dateSelect`, `departmentList`, `users`, `logs`, `pageNumber`, `totalRecords`, `pageSize`, `searchValue`, `statusFilter`, `loading`, `dateRange`, `selectedUser`, `selectedDepartment`
Derived	`useMemo`	—
Effects	Multiple `useEffect`	Fetch departments on mount; fetch users when department changes; fetch logs when `dateRange`, `selectedUser`, `selectedDepartment`, `statusFilter` change; count total logs

---

Effects & Dependencies

- Line 210-250: `fetchDepartments` — runs once on mount
- Line 252-292: `fetchUsers` — dependency: `[selectedDepartment]` — fetches users for department filter
- Line 294-360: `logsData` — **main data fetch**; dependency: `[dateRange, selectedUser, selectedDepartment, statusFilter]` — fetches daily logs list with pagination
- Line 362-400: `handleStatusUpdate` — on confirm, updates log status via API, refreshes logs list
- Line 402-445: `handleDownload` — dependency: `[dateRange, selectedUser, selectedDepartment]` — downloads Excel

UI Components Used (tree)

```
DailyLogs
├── ReusableTable (component, ../components/table) — daily logs table w
├── Tabs, Tab (from @heroui/react) — status filter tabs (Active/Complet
├── Button (from @heroui/button) — download, status update actions
├── CustomDateRangePicker (component, ../components/CustomDateRangePick
├── Autocomplete, AutocompleteItem (from @heroui/react) — department fi
├── Select, SelectItem (from @heroui/select) — status filter selection
├── Format from date-fns — date formatting
├── AddToast (from @heroui/react) — notifications
├── title from primitives — page title
```

Known Defects, Dead Code, TODOs

1. **Line 34**: `isAdminOrTech` guard: `user.userRole === 1 || user.departmentID === 24` — hardcoded role and department IDs. If backend

changes these, access control will break.
2. **Lines 85-95**: Date range default logic — same pattern as other pages; if `selectedUser` set → no date range; otherwise default 7 days.
3. **Status filter logic**: `statusFilter` state initialized and used to filter logs; if backend changes status values (e.g., from "ACTIVE" to "ACTIVE_LOG"), the filter won't work correctly.
4. **`selectedUser` special handling**: `isELORUSER ? String(user?.userID) : ""` — hardcoded role checking passed to API payload.
5. **`selectedDepartment` handling**: Similar pattern — `String(selectedDepartment ?? "")` — if department ID changes format, may cause issues.
6. **`statusFilter` initialization**: `useState("ALL")` — hardcoded default; if backend changes status constants, default may show wrong data.
7. **Duration field in table**: Raw duration value displayed — if backend changes duration unit (minutes vs hours), display will be misleading.
8. **`handleStatusUpdate` payload**: `logID` and `status` — if backend changes the expected payload format, status updates will fail.
9. **Multiple `console.log` statements** left in the code.
10. **Download endpoint** `/download-daily-logs`: Same concerns as other download endpoints about hardcoded API base URL.

Backend Migration Impact

High risk — this page makes direct API calls to daily logs endpoints:

API endpoints:

- `/Department/list` — department filter
- `/users/read` (implied) — user filter
- `/daily-logs/list` — **CRITICAL**: Paginated list of daily logs; response has `logs` array with `logID`, `userID`, `date`, `activityType`, `status`, `duration`, `createdAt`
- `/daily-logs/count` — returns total count for pagination
- `/daily-logs/update-status` — updates log status; payload: `logID`, `status`
- `/download-daily-logs` — downloads Excel; payload: `departmentID`, `userID`, `startDate`, `endDate`

Key concerns:

1. **`isAdminOrTech` guard**: `user.userRole === 1 || user.departmentID === 24` — hardcoded role (1) and department (24). If backend changes role IDs or department IDs, access control will break — admins or tech team may be locked out or unauthorized users may gain access.
2. **Status values**: Hardcoded `ACTIVE`, `COMPLETED`, `CANCELLED`, `ALL` — if backend

changes status constants, the status filter UI and API calls will misbehave.
3. **Duration unit**: The `duration` field displayed in table — if backend changes unit (e.g., from minutes to hours, or adds new unit types), the display will be incorrect without frontend conversion.
4. **Date range handling**: Same `convertToZonedFormat`, `normalizeISO`, `formatDateTime` patterns — timezone differences between Java/Spring Boot and Node.js may cause logs to show for wrong date periods.
5. **`selectedUser` and `selectedDepartment`**: Both use `String(...)` conversion passed to API — if user/dept object shapes change after login, wrong IDs may be sent.
6. **`isELORUSER` pattern**: Used for determining if user should pass their ID to API — same hardcoded role checking pattern seen across many pages.

HR Feedback Page (src/pages/hrFeedback.tsx)

Route: `/hr-feedback`
Access: `adminOnly` guard (requires `user.userRole === 1`)
File: `src/pages/hrFeedback.tsx` (342 lines)

Purpose & User Flow

HR Feedback page — allows admin users to view and manage HR feedback entries. Features:

- View feedback list with filtering
- Department and user filtering
- Date range picker
- Status filtering (Pending, Resolved, etc.)
- Reply to feedback entries
- Update feedback status
- Export/download functionality

User flow:

1. Admin lands on HR feedback page
2. Can filter by department (autocomplete) and/or user (autocomplete)
3. Date range determines which feedback period
4. Table shows feedback entries with: ID, user, department, subject, message, status, created date
5. Can select a feedback entry to reply or update status

6. "Reply" opens modal with textarea and status selector
7. "Download" exports feedback as Excel

API Calls

#	Method	Endpoint	Request Payload	Response Fields Used	W
1	POST	/t2.0/authentificationUtilization/Department/list?page=0&size=0	searchText, filter, startDate, endDate, roleID: user.userRole, userID: ...	Department list: departmentID, departmentName	On mou
2	POST	/t2.0/authentificationUtilization/hr-feedback/list?page=&size=	departmentID, userID, startDate, endDate, page, size, status	Feedback list: feedbackID, userID, departmentID, subject, message, status, createdAt, respondedAt, repliedBy	When p pageSi dateRa select select status changes
3	POST	/t2.0/authentificationUtilization/hr-feedback/count	departmentID, userID, startDate, endDate, status	Total count for pagination	When fi
4	POST	/t2.0/authentificationUtilization/hr-feedback/update-status	feedbackID, status	Success response	When s confirm
5	POST	/t2.0/authentificationUtilization/hr-feedback/reply	feedbackID, replyText, respondedBy	Success response	When "S confirm

---

Special Logic

Logic	Description

Status filter options (lines 31-35):	`PENDING`, `RESOLVED`, `ALL` — toggles which feedback shows
`handleStatusUpdate` (lines 180-200):	Confirms with user, then calls `/update-status` API with `feedbackID` and new `status`
`handleReply` (lines 210-235):	Confirms with user, gets reply text, calls `/reply` API with `feedbackID`, `replyText`, and `respondedBy` (from auth user)
Date range default (lines 85-95):	If `selectedUser` set → no date range; otherwise default 7 days from today
`isAdminOnly` check (line 34):	`user.userRole === 1` — only admins can access HR feedback

---

State

Type	Source	Description
Local state	Multiple `useState`	`sidebarOpen`, `dateSelect`, `departmentList`, `users`, `feedbackList`, `pageNumber`, `totalRecords`, `pageSize`, `searchValue`, `statusFilter`, `loading`, `dateRange`, `selectedUser`, `selectedFeedback`, `replyText`, `isReplyModalOpen`
Derived	`useMemo`	—
Effects	Multiple `useEffect`	Fetch departments on mount; fetch users when department changes; fetch feedback when `dateRange`, `selectedUser`, `selectedDepartment`, `statusFilter` change; count total feedback; handle reply/submit effects

---

Effects & Dependencies

- Line 195-235: `fetchDepartments` — runs once on mount
- Line 237-277: `fetchUsers` — dependency: `[selectedDepartment]` — fetches users for department filter
- Line 279-340: `feedbackData` — **main data fetch**; dependency: `[dateRange, selectedUser, selectedDepartment, statusFilter]` — fetches HR feedback list with pagination
- Line 342-380: `handleStatusUpdate` — on confirm, updates feedback status via API, refreshes feedback list
- Line 382-420: `handleReply` — on confirm, opens reply modal; on submit, calls `/reply` API, closes modal, refreshes list
- Line 422-465: `handleDownload` — dependency: `[dateRange, selectedUser,`

`selectedDepartment]` — downloads Excel

UI Components Used (tree)

```
HRFeedback
├── ReusableTable (component, ../components/table) — HR feedback table
├── Tabs, Tab (from @heroui/react) — status filter tabs (Pending/Resolv
├── Modal (from @heroui/react) — reply modal with textarea and status s
├── ModalContent, ModalHeader, ModalBody, ModalFooter (from @heroui/rea
├── useDisclosure (from @heroui/react) — for reply modal
├── Button (from @heroui/button) — download, reply, status update actio
├── CustomDateRangePicker (component, ../components/CustomDateRangePick
├── Input, Textarea (from @heroui/input, @heroui/textarea) — reply text
├── Autocomplete, AutocompleteItem (from @heroui/react) — department fi
├── Select, SelectItem (from @heroui/select) — status filter selection
├── Format from date-fns — date formatting
├── AddToast (from @heroui/react) — notifications
├── title from primitives — page title
```

Known Defects, Dead Code, TODOs

1. **Line 34**: `isAdminOnly` guard: `user.userRole === 1` — hardcoded role ID. If backend changes role IDs, access control will break.
2. **Lines 85-95**: Date range default logic — same pattern as other pages; if `selectedUser` set → no date range; otherwise default 7 days.
3. **Status filter logic**: `statusFilter` state initialized and used to filter feedback; if backend changes status values (e.g., from "PENDING" to "OPEN"), the filter won't work correctly.
4. **`selectedUser` special handling**: `isELORUSER ? String(user?.userID) : ""` — hardcoded role checking passed to API payload.
5. **`selectedDepartment` handling**: Similar pattern — `String(selectedDepartment ?? "")` — if department ID changes format, may cause issues.
6. **`statusFilter` initialization**: `useState("ALL")` — hardcoded default; if backend changes status constants, default may show wrong data.
7. **Reply textarea**: No character limit validation — users could submit very long replies; if backend has limits, may be truncated or rejected.
8. **`handleReply` payload**: `feedbackID`, `replyText`, `respondedBy` — if backend changes expected payload format (e.g., different field names), replies will fail.
9. **`respondedBy`**: Always set from authenticated user — if user object shape changes after login, may send wrong value.

10. **Multiple `console.log` statements** left in the code.
11. **Download endpoint** `/download-feedback` or similar: Same concerns as other download endpoints about hardcoded API base URL.
12. **Reply modal always opens with empty `replyText`**: State initialized to `""` — if user navigates to different feedback then returns, reply text may not persist as expected.

Backend Migration Impact

High risk — this page makes direct API calls to HR feedback endpoints:

API endpoints:

- `/Department/list` — department filter for HR feedback
- `/users/read` (implied) — user filter
- `/hr-feedback/list` — **CRITICAL**: Paginated list of HR feedback entries; response has `feedback` array with `feedbackID`, `userID`, `departmentID`, `subject`, `message`, `status`, `createdAt`, `respondedAt`, `repliedBy`
- `/hr-feedback/count` — returns total count for pagination
- `/hr-feedback/update-status` — updates feedback status; payload: `feedbackID`, `status`
- `/hr-feedback/reply` — submits reply to feedback; payload: `feedbackID`, `replyText`, `respondedBy`

Key concerns:

1. **`isAdminOnly` guard**: `user.userRole === 1` — hardcoded role ID; if backend changes role names/IDs, admin access will be broken.
2. **Status values**: Hardcoded `PENDING`, `RESOLVED`, `ALL` — if backend changes status constants, the status filter UI and API calls will misbehave.
3. **`respondedBy` field**: Always populated from authenticated user — if backend changes how "responded by" is stored or if the format changes, reply tracking will break.
4. **Subject/Message fields**: The `subject` and `message` fields displayed in table — if backend changes these field names or adds new fields (e.g., `priority`, `category`), the UI will show incorrect or missing data.
5. **Date range handling**: Same `convertToZonedFormat`, `normalizeISO`, `formatDateTime` patterns — timezone differences between Java/Spring Boot and Node.js may cause feedback to show for wrong date periods.
6. **`selectedUser` and `selectedDepartment`**: Both use `String(...)` conversion passed to API — if user/dept object shapes change after login, wrong IDs may be sent.
7. **`isELORUSER` pattern**: Used for determining if user should pass their ID to API — same hardcoded role checking pattern seen across many pages.

Task Management Page (src/pages/TaskManagement.tsx)

Route: `/task-management`
Access: All authenticated users (Download button hidden for users with `departmentID`)
File: `src/pages/TaskManagement.tsx` (731 lines)

Purpose & User Flow

Task management — create, edit, activate/deactivate standard tasks assigned to departments. Features:

- Task list with search, filtering (frequency, status), sorting, pagination
- Add New Task modal with full form
- Edit Task via row action
- Activate/Deactivate single task with confirmation modal
- Download All button (disabled for users with departmentID)
- Bulk selection with activate/deactivate toolbar

User flow:

1. User lands on task list showing all tasks with pagination
2. Can search tasks by name/description, filter by frequency or active status
3. "Add New Task" → modal with task name, description, department, task type, standard time, frequency, dynamic flag, role
4. Edit action on row → pre-populates same modal
5. Select one or more rows → toolbar shows Activate/Deactivate buttons
6. Confirm modal before status toggle
7. Download All button available only for admin-level users (no departmentID)

API Calls

#	Method	Endpoint	Request Payload	Response Fields Used	When I

1	POST	/Department/list?page=0&size=	searchText: "", filter: "", startDate: "", endDate: "", roleID: 1, userID: 1	response.departments[]: departmentID, departmentName, departmentDescription, newStatus	On mour (once)
2	POST	/departmentTasks/list?page=&size=	searchText: searchValue, filter: "", startDate: "", endDate: ""	response.tasks[]: taskID, taskName, description, departmentID, departmentName, standardTimeMins, frequency, isDynamic, isActive, createdAt, modifiedAt, timingType, roleID; response.totalRecords	When pageSiz pageNun searchV change
3	POST	/departmentTasks/create	taskName, description, departmentID, standardTimeMins, frequency, isDynamic, timingType, roleID	data.data (new task object)	When "A Task" submitte
4	PUT	/departmentTasks/update/{id}	Same as create	data.data (updated task)	When "U Task" submitte
5	PUT	/departmentTasks/{taskID}/status?isActive={bool}	{} (empty body; status via query param)	—	When Activate/ Deactiva confirme

Note: Bearer token read from `localStorage.authUserLD` for create/update/toggle. Department list and task list calls send only `Content-Type` header (no auth token).

Special Logic

Logic	Description

Department list hardcoded params (lines 240-241)	`roleID: 1, userID: 1` — always fetches as admin regardless of actual user
Task list null filter (line 285)	`.filter((t: any) => t != null)` — filters out null/undefined entries from API response
`createdAt.split("T")[0]` (line 288)	Extracts date portion from ISO timestamp
`timingType` → `type` mapping (line 298)	API field `timingType` mapped to frontend field `type`
Department select filters Active only (line 548)	`departmentList.filter(ele => ele.status === "Active")`
Task type hardcoded list (lines 562-599)	37 task types: "Fixed", "Each Client", "Each Deal", "Each Query", ..., "Each Waterfall", "Each WF"
Role select hardcoded IDs (lines 663-670)	`1=Root User, 2=User, 3=EL, 4=FL, 5=Intern, 6=Executive, 7=Manager, 23=Lead`
Download button disabled (lines 374-378)	`isDisabled={loading || !!authUserLD?.departmentID}` — only users without departmentID can download
Toggle status via query param (line 327)	`PUT /departmentTasks/{id}/status?isActive={bool}` — status passed as query string, not body

---

Forms

Add/Edit Task Modal

Field	Type	Validation	Default	Notes
Task Name	Input	—	`''`	Required for meaningful task
Description	Textarea	—	`''`	—
Department	Select	—	`''`	Only Active departments shown; uses separate `selectedDepartment` state
Task Type	Select	—	`''`	37 hardcoded options (lines 562-599)
Standard Time (mins)	Input (text)	—	`30`	Sent as `Number(...)` in payload; stored as string in state
Frequency	Select	—	`'Daily'`	Options: Daily, Weekly, Monthly, Dynamic, Not Daily
Is Dynamic	Select	—	`false`	Yes/No (`'true'/'false'` string comparison)

Role	Select	—	`''`	Hardcoded role IDs: 1,2,3,4,5,6,7,23

Submit behavior: Calls `handleAddTask` (create) or `handleEditTask` (update); on success: toast, append/replace in `tasks` state, close modal, reset form. On failure: toast with `error?.response?.data?.message`, close modal.

Form reset issue: `setNewTask` reset (lines 100-108, 159-167) omits `type` field — will retain previous task type after reset.

Tables/Lists

Table	Columns	Sorting	Filtering	Pagination	Export/ Download
Task list (ReusableTable)	Created At (formatted `dd MMM yyyy`), Task Name, Department ID, Department Name, Standard Time (mins), Frequency, Dynamic, Status	✓ (per-column `sortable: true`)	✓ (Frequency: Daily/Weekly/ Monthly/ Dynamic/Not Daily; Status: Active/ Inactive)	✓ (`pageSize` default 10)	Download All button (disabled for dept users)

Table actions:

- Edit → opens modal with row data pre-populated

Filter options (lines 481-502):

- `frequency`: Daily, Weekly, Monthly, Dynamic, Not Daily
- `isActive`: true (Active), false (Inactive)

Bulk actions toolbar (lines 407-457): Appears when `selectedRows.length > 0`; Activate and Deactivate buttons; each validates exactly 1 row selected.

State

Type	Source	Description

Local state	Multiple `useState`	`sidebarOpen`, `loading`, `selectedDepartment`, `departmentList`, `SelectedTask` (note: capital S), `isEditing`, `selectedRows`, `confirmOpen`, `toggleStatusValue`, `tableRefreshKey`, `pageNumber`, `totalRecords`, `pageSize`, `searchValue`, `tasks`, `isModalOpen`, `newTask`
Auth	`localStorage.authUserLD`	Read inside component (line 180); `authUser` parsed but only `token` used
Effects	Two `useEffect`	Departments on mount; tasks when `authUserLD`, `pageSize, pageNumber, searchValue` change

---

Effects & Dependencies

- Lines 232-267: `fetchDepartments` — runs once on mount; **no auth token** in headers; hardcoded `roleID: 1, userID: 1`
- Lines 268-311: `fetchTasks` — dependency: `[authUserLD, pageSize, pageNumber, searchValue]`; **no auth token** in headers; null-filters response; maps `timingType` → `type`

UI Components Used (tree)

```
TaskManagement
├── ReusableTable (component, ../components/table) — main task list
├── Modal, ModalContent, ModalHeader, ModalBody, ModalFooter (from @her
├── addToast (from @heroui/react) — notifications
├── Button (from @heroui/button) — Add New Task, Download All, Activate
├── Input (from @heroui/input) — Task Name, Standard Time
├── Textarea (from @heroui/input) — Description
├── Select, SelectItem (from @heroui/select) — Department, Task Type, F
├── Sidebar, Topbar (imported but NOT rendered in JSX) — dead imports
├── title (from ../components/primitives) — page title
├── format (from date-fns) — `dd MMM yyyy` for Created At column
├── RollerCoaster (from lucide-react) — imported but NOT used — dead im
├── axios — API calls
```

Known Defects, Dead Code, TODOs

1. **Line 14**: `RollerCoaster` imported from lucide-react but never used — dead import

2. **Lines 9-10**: `Sidebar` and `Topbar` imported but not rendered — dead imports
3. **Line 7**: Commented out `// import { addToast } from "@heroui/react"` — dead comment
4. **Line 36**: `SelectedTask` with capital S (unused after `setSelectedTask(row)` on edit) — inconsistent naming
5. **Lines 240-241**: Department fetch hardcodes `roleID: 1, userID: 1` — always queries as admin; if backend uses these for permission filtering, non-admin users get wrong department list
6. **Lines 244-248, 278-282**: Department and task list calls send **no Authorization header** — only create/update/toggle do (lines 76-79, 133-136, 320-324)
7. **Lines 100-108, 159-167**: Form reset after create/update omits `type` field — `newTask.type` retains previous value
8. **Line 617**: `standardTimeMins` stored as string in state (`e.target.value`) but typed as `number` in interface — sent as `Number(...)` in payload
9. **Lines 120, 168**: `handleEditTask(id)` parameter `id` is untyped (implicit `any`) — line 682 passes `newTask.taskId`
10. **Line 284**: `data.response.tasks` accessed without null check on `data.response` — will crash if response shape changes
11. **Line 288**: `t.createdAt.split("T")[0]` — will crash if `createdAt` is null/undefined
12. **Lines 374-378**: Download button uses double-negative logic: `className={... !authUserLD?.departmentID ? "" : "opacity-0..."}` AND `isDisabled={... !!authUserLD?.departmentID}` — visually hidden AND disabled for dept users
13. **Line 327**: Status toggle uses query parameter `?isActive=${newStatus}` — if backend switches to body param, toggle breaks
14. **Line 36**: `SelectedTask` state set but never read — dead state
15. **Lines 663-670**: Role select hardcodes IDs `[1,2,3,4,5,6,7,23]` — same pattern as access-management
16. **Lines 562-599**: 37 hardcoded task type options — if backend adds/removes types, UI won't reflect changes
17. **Multiple `console.log` statements** (lines 88, 145, 211, 258)
18. **Line 106**: Reset object includes `departmentID: ""` but uses `selectedDepartment` state for payload — departmentID in newTask is effectively unused for API calls

---

Backend Migration Impact

High risk — task CRUD endpoints with unique conventions:

API endpoints:

- `/Department/list` — department dropdown; hardcoded `roleID: 1, userID: 1`

and `size=` (empty, not 0)
- `/departmentTasks/list` — paginated task list; `searchText` only filter; response has `tasks[]` and `totalRecords`
- `/departmentTasks/create` — 8-field payload: `taskName`, `description`, `departmentID`, `standardTimeMins`, `frequency`, `isDynamic`, `timingType`, `roleID`
- `/departmentTasks/update/{id}` — same 8 fields; PUT method
- `/departmentTasks/{id}/status?isActive={bool}` — PUT with **empty body**; status via query parameter — unusual pattern

Key concerns:

1. **Query-param status update** (line 327): If Node.js backend expects body instead of query param, activate/deactivate breaks
2. **No auth on GET endpoints**: Department list and task list calls have no Bearer token — if backend adds auth middleware to these, they'll fail
3. **`timingType` field**: API returns `timingType` but frontend maps to `type` — if backend renames, the edit form won't pre-populate task type
4. **37 hardcoded task types**: UI-only list; if backend validates types and they diverge, create/update fails
5. **Hardcoded `roleID: 1, userID: 1`** in department fetch: If backend uses these for data scoping, non-admin users see wrong departments
6. **Role IDs `[1,2,3,4,5,6,7,23]`**: Same hardcoded mapping as access-management — breaks if backend renumbers roles
7. **`response.tasks` null-filter** (line 285): Frontend defends against null entries in array — if backend sends nulls differently (undefined vs null), filter may not catch them

---

Cross-Cutting Concerns (PHASE 3)

Hardcoded Values Summary

Category	Values	Locatio

Role IDs	1 = Root User, 2 = User, 3 = EL, 4 = FL, 5 = Intern, 6 = Executive, 7 = Manager, 23 = Lead	`roleMap` object pages; `isReadO` checks `[2,5,6,` `adminOnly` `===` `salesOnly` `departmentID` `HrOnly` `departmentID` `TechOnly` `departmentID` `adminOrTech` `role===1`
Department IDs	21 = Sales, 22 = HR, 24 = Tech	`salesOnly` `departmentID` `HrOnly` `departmentID` `TechOnly` `departmentID` `isReportUser` `departmentID` `adminOrTech` `dept===24`
Status Codes	1 = Active, 0 = Inactive (toggle); ACTIVE/COMPLETED/CANCELLED/PENDING/RESOLVED (text)	`toggleUserSt` `userStatus: 1`
Date Cutoff	`2026-06-30`	`08-goals.md` l `04-efficienc` 526; `14-query-` `report.md` line multiple pages
API Base URL	`https://taskly.tapmind.io/t2.0/authentificationUtilization`	ALL 60+ API ca relative paths ag base

Check	Pages	Count
`user.departmentID === 21`	11-sales-forward, 14-query-report	2 pages
`user.departmentID === 22`	10-access-management	1 page
`user.departmentID === 24`	17-daily-logs, 18-hr-feedback, 14-query-report, `adminOrTech`	4 pages
`user.departmentID === 16`	12-pms.md	1 page

Auth/Security Review

Critical Security Concerns

1. **JWT stored in localStorage** (`authUserLD`) — XSS vulnerability; no HttpOnly cookie; if XSS exists, attacker can steal tokens
2. **No refresh token logic** — when JWT expires, user must re-login; long-running sessions force logout
3. **All API calls use same hardcoded base URL** — if base URL compromised, all endpoints accessible
4. **No rate limiting visible** — API calls from frontend may be vulnerable to abuse
5. **`window.location.reload()` after delete** (line 177 in access-management, similar in other pages) — may cause resubmission on refresh; no PRG (Post/Redirect/Get) pattern
6. **No CSRF tokens** — since using localStorage JWT, vulnerable to CSRF if site doesn't implement double-submit cookie pattern
7. **`authUserLD` null checks inconsistent** — some components check `!!authUserLD?.userRole`, others assume it exists; may cause runtime errors
8. **`setAllUser` called in multiple places** — `commonStore.tsx`; potential race conditions if multiple fetches update store simultaneously

State Management Issues

- **Zustand persisted store** (`commonStore.tsx`) stores auth state in localStorage — if cleared, user logged out unexpectedly
- **No centralized API client** — each page manages its own API calls; inconsistent error handling; some show toast on error, some don't
- **Multiple `useEffect` fetch patterns** — some on mount with dependencies, some conditional; hard to track data freshness
- **`setAllUser` side effect** in `commonStore.tsx` — called from multiple page mounts;

may overwrite user data unexpectedly

Frontend-Backend Logic Duplication

What Frontend Does That Should Be Backend

Feature	Frontend Implementation	Should Be Backend
Role-based access control	12+ pages have `adminOnly`, `salesOnly`, `HrOnly`, `TechOnly`, `adminOrTech` guards hardcoded	Centralized auth middleware in backend
Status code mapping (1/0 for Active/ Inactive)	`handleToggleUserStatus` etc.	Auth/status service
Efficiency level calculation (High/At Par/Low)	`04-efficiency.md` lines 471-486; `01-dashboard`; `04-efficiency`; `14-query-report`; `15-reports`	Business logic service
Filter field/operator/ value mapping	`14-query-report` lines 376-381; `11-sales-forward` validate()	Query builder service
Date range normalization	`convertToZonedFormat`, `normalizeISO`, `formatDateTime` patterns everywhere	Date/time service in backend
Pagination empty state (`page=0&size=0`)	Multiple pages assume empty params = all records	Backend pagination defaults

What Backend Does That Frontend Replicates

Feature	Backend Should Provide	Frontend Currently Does
Permission checking	Role/dept checks in auth middleware	`isReadOnly`, `isReportUser`, `pmsPermissions` from localStorage
Filter options generation	Department/user lists with proper formatting	`fetchDepartments`, `fetchUsers` in every page
Summary computations	Avg calculations, efficiency levels	Frontend reduces API data into summary cards
Download/Excel generation	Backend generates Excel, frontend just calls endpoint	All pages call `/download-...` endpoints

---

Consolidated Defects Across All Pages

Defect Type	Count	Description
Hardcoded role IDs	20+ occurrences	`user.userRole === 1` or specific role numbers
Hardcoded department IDs	10+ occurrences	`departmentID === 21, 22, 24, 16`
Hardcoded status values	8+ occurrences	`ACTIVE`, `PENDING`, `RESOLVED`, `1`, `0`, etc.
Hardcoded date cutoffs	4 occurrences	`2026-06-30` in goals, efficiency, query-report, reports
Inconsistent `selectedUser` handling	6+ occurrences	`isELORUSER ? String(user?.userID) : ""` pattern
Inconsistent `selectedDepartment` handling	5+ occurrences	`String(selectedDepartment ?? "")` pattern
Missing null checks on auth	4+ occurrences	`authUserLD` may be null but code assumes exists
Duplicate `console.log` statements	15+ across codebase	Debug left in production
`window.location reload()` after mutations	3+ occurrences	Not PRG pattern; causes resubmission
Commented-out password field	1 occurrence	Access management form
Type mismatches (Task[] vs call objects)	2 occurrences	Sales forward, access management

---

Migration Recommendations

High Priority (Must Fix Before Migration)

1. **Extract role/department checks to backend middleware** — remove all `adminOnly`, `salesOnly`, `HrOnly`, `TechOnly`, `adminOrTech` from frontend; let backend handle authorization and return only authorized data
2. **Replace hardcoded API base URL** with environment variable or build-time config — currently all 60+ calls break if URL changes
3. **Implement refresh token flow** or session management — currently no way to get new JWT without re-login

4. **Centralize efficiency level calculation** in backend service — frontend should just receive the level, compute locally
5. **Create shared API client** (Axios instance with interceptors) — consistent error handling, auth token injection, refresh logic

Medium Priority (Should Fix)

6. **Replace hardcoded status values** with enums or constants from backend — frontend should use backend-provided status options
7. **Implement proper PRG pattern** after mutations — replace `window.location.reload()` with state update + redirect
8. **Add null checks for authUserLD** throughout — or ensure auth state is always populated before these components render
9. **Remove duplicate `console.log` statements** — clean up before migration
10. **Create role/department enum types** — frontend should use backend-provided role names/IDs, not hardcoded numbers

Low Priority (Nice to Have)

11. **Implement i18n support** — currently all English-only; if migration involves multi-language support, i18n should be added
12. **Add proper type safety** — replace `any` types, fix `Task[]` vs call object mismatches, add proper interfaces
13. **Create unit tests for cross-cutting concerns** — ensure role guards, filter logic work correctly
14. **Add loading/error state consistency** — currently some pages show skeleton, some show nothing, some toast on error

---

API Contract Map (PHASE 4)

All frontend API calls against the Java/Spring Boot backend. Base URL is hardcoded as `https://taskly.tapmind.io/t2.0/authentificationUtilization`.

Master Endpoint Table

#	Method	Endpoint	Pages Using	Reque
1	POST	`/Department/list?page=0&size=0`	01,03,04,05,06,07,08,09,10,14,15,16,17,18	searchTe startDat roleID, u

14	POST	`/download-efficiency-summary`	01,04,09,14,15	Same as
15	POST	`/save-targets`	16	targets[{ userID, t
16	POST	`/daily-logs/list?page=&size=`	17	departm startDat page, siz
17	POST	`/daily-logs/count`	17	departm startDat
18	POST	`/daily-logs/update-status`	17	logID, st
19	POST	`/download-daily-logs`	17	departm startDat
20	POST	`/hr-feedback/list?page=&size=`	18	departm startDat page, siz
21	POST	`/hr-feedback/count`	18	departm startDat status
22	POST	`/hr-feedback/update-status`	18	feedback
23	POST	`/hr-feedback/reply`	18	feedback respond
24	POST	`/sales/calls/list?page=&size=`	11	userID
25	POST	`/sales/calls/reschedule`	11	userID, newDate reschedu
26	POST	`/sales/calls/update-status`	11	userID, newDate
27	POST	`/departmentTargets/getTargetsByDepartment`	09	departm
28	POST	`/departmentGoals/getGoalsByDepartment`	09	departm

29	POST	`/goal/create`	07	goal det
30	POST	`/goal/update`	07	goal det
31	POST	`/goal/delete`	07	goalID
32	POST	`/userGoalData`	08	userID, s endDate
33	POST	`/task/list?page=&size=`	01,02,04,05,06	filters, p
34	POST	`/entry-approval/list`	05	departm startDat status
35	POST	`/entry-approval/approve`	05	entryID,
36	POST	`/entry-approval/reject`	05	entryID, reason
37	POST	`/login`	Auth flow	email, p
38	POST	`/goal-management/list`	07	departm startDat
39	POST	`/departmentTasks/list?page=&size=`	19	searchTe startDat
40	POST	`/departmentTasks/create`	19	taskNam departm standard frequenc timingTy
41	PUT	`/departmentTasks/update/{id}`	19	Same as
42	PUT	`/departmentTasks/{taskID}/status?isActive={bool}`	19	{} (empt via quer

Response Shape Concerns

`/users/read` — Dual Response Shape

```
Shape A: { user: { departmentID: 1 } }          — single department
Shape B: { user: { departments: [{...}, ...] } } — multiple departments
```

Frontend handles both via `getUserDepartments()` in 06-forms, 13-primary-department, and similar normalization in 10-access-management, 14-query-report. **If Node.js backend standardizes to one shape, all normalization code breaks or becomes dead code.**

`/list-efficiency-summary` — Summary Object

Every efficiency page depends on exact field names:

- `AvgTargetPercent`, `AvgUtilizationPercent`, `AvgTotalWork`, `AvgGoalPercent`, `AvgAllocatedHrs`
- Data array items with same fields per department/user

Renaming any field breaks 8+ pages simultaneously.

Pagination Convention

Frontend sends `page=0&size=0` to mean "fetch all". If Node.js backend treats size=0 as "return nothing", all lists break.

Endpoints NOT Called by Frontend (Backend-Only)

- Background job endpoints
- Push notification registration
- Admin-only backend operations
- Reporting/analytics aggregation jobs

These must continue working during migration but require no frontend changes.

UI Changes Required (PHASE 5)

Changes needed per page/module during the Java→Node backend migration.

Global Changes (Affect All Pages)

#	Change	Reason	Files Affected	R
G1	Replace hardcoded API base URL with env var (`VITE_API_BASE_URL`)	Backend URL may change during migration; currently 60+ calls hardcode `https://taskly.tapmind.io/t2.0/authentificationUtilization`	ALL pages, ALL components with API calls (60+)	**HIGI** singl point failu
G2	Create shared Axios instance with interceptors (auth token injection, refresh, error handling)	Currently each page manages auth headers and errors independently	New file `src/lib/api.ts` + all pages	**HIGI** refac 60+ sites
G3	Add refresh token or session-expiry handling	Currently no refresh logic; JWT expiry forces silent API failures	`src/lib/api.ts`, auth store	**HIGI**
G4	Standardize error handling (toast on all API failures)	Some pages show toasts, others silently fail	All pages	Medi
G5	Extract role/dept checks to backend-returned permissions	20+ hardcoded `userRole === 1`, `departmentID === 21/22/24` checks	ProtectedRoute, 18+ pages	**HIGI**
G6	Remove `window.location.reload()` after mutations; use state updates	PRG pattern missing; causes form resubmission on refresh	10-access-management, 03-department, others	Medi

Per-Page Changes

01-dashboard.tsx

#	Change	Reason	Risk
D1	Adapt efficiency summary field names if backend renames	6 summary fields consumed	Medium
D2	Handle 2026-06-30 cutoff removal if backend consolidates paths	Hardcoded date logic	Low

03-department.tsx + 13-primary-department

#	Change	Reason	Risk
P1	Standardize user shape response	Dual `{departmentID}` vs `{departments[]}` shapes	Medium
P2	Update role ID references in roleMap	Hardcoded `1: "Root User"` etc.	Medium

05-entry-approval.tsx

#	Change	Reason	Risk
E1	Confirm approve/reject payload field names	`entryID`, `approvedBy`, `rejectedBy` may change	High
E2	Status value constants alignment	Status strings hardcoded	Medium

06-forms.tsx

#	Change	Reason	Risk
F1	Update `getUserDepartments()` normalization	Dual shape handling	Medium

07-goal-management.tsx

#	Change	Reason	Risk
G1	Goal CRUD payload field names	create/update/delete endpoints	High
G2	Role-based access logic (adminOrTech etc.)	Hardcoded role IDs	Medium

08-goals.tsx

#	Change	Reason	Risk
GL1	Remove/replace 2026-06-30 hardcoded cutoff	Date-specific branching breaks after cutoff	**HIGH**
GL2	Handle dual API paths (efficiency summary vs userGoalData)	Backend may consolidate	Medium

09-history.tsx

#	Change	Reason	Risk

H1	Efficiency level computation → move to backend	Frontend duplicates business logic	Medium
H2	Department targets/goals endpoint payloads	`departmentId` field naming	Medium

10-access-management.tsx

#	Change	Reason	Risk
A1	Update 13-field user create/update payload	Any field rename breaks user management	**CRITICAL**
A2	Fix status polarity (1/0) with backend	Hardcoded `userStatus: 1	0`
A3	Fix `roleFilter` null initialization (`Number(null)=0`)	May fetch wrong role on first load	Medium
A4	Password field commented out — re-enable or remove	Line 718-730 dead code	Low
A5	Remove hardcoded roleMap `[1,3,4,2,5,6,7,23]`	Role IDs may change	High

11-sales-forward.tsx

#	Change	Reason	Risk
S1	Status value constants (6 statuses hardcoded)	Status strings may change	High
S2	Cancelled-by inconsistency (`PUBLISHER` vs `CANCELLED_BY_PUBLISHER`)	Variable naming mismatch	Medium
S3	Timezone handling in `convertToZonedFormat`	Node.js backend timezone may differ	Medium
S4	Fix `Task[]` type on call data	Type mismatch	Low

12-pms.tsx

#	Change	Reason	Risk
PM1	Replace localStorage permissions with API-fetched permissions	`localStorage.roles` format may change	High
PM2	Fix hardcoded `departmentID === 16` for report access	Dept ID may change	Medium

PM3	Fix hardcoded read-only roles `[2,5,6,23]`	Role IDs may change	Medium

14-query-report.tsx

#	Change	Reason	Risk
Q1	Advanced filter field names alignment	`Target Achieved`, `Goal Achieved`, etc. must match backend	High
Q2	Remove 2026-06-30 cutoff	Same as GL1	**HIGH**
Q3	Remove dept 7 / user 90 special path	Lines 530-533 hardcoded	Medium
Q4	Fix inverted dept filter logic (`user.departmentID ? null : ...`)	Confusing UX	Low

15-reports.tsx

#	Change	Reason	Risk
R1	Three summary endpoints field alignment	efficiency/target/utilization summaries	High
R2	Remove hardcoded cutoff date	Same as GL1	**HIGH**
R3	Fix `adminOrTech` hardcoded guard	`role===1	

16-target-management.tsx

#	Change	Reason	Risk
T1	`save-targets` payload alignment	3-field array structure	High
T2	Input validation (no min/max on %)	Invalid values may be sent	Medium
T3	`adminOnly` hardcoded guard	`userRole === 1`	High

17-daily-logs.tsx

#	Change	Reason	Risk
DL1	Status constants alignment (ACTIVE/ COMPLETED/CANCELLED)	Filter mismatch if backend changes	High
DL2	Duration unit display (minutes vs hours)	Display accuracy	Medium
DL3	`adminOrTech` hardcoded guard	`role===1	

- **AI assistance** for code generation, refactoring, testing (reduces ~40% manual effort)
- Backend migration runs **in parallel** (separate team/person)
- Frontend must keep working throughout migration
- Working days only (no weekends/holidays)
- Starting from a clean slate with this audit as reference

Task Breakdown

Phase A: Foundation (Week 1-2) — 10 working days

#	Task	Est. Days	Dependencies
A1	Create shared API client (`src/lib/api.ts`) with interceptors, base URL from env	1.5	—
A2	Replace all 60+ hardcoded API URLs with shared client	2	A1
A3	Add auth token injection + 401 handling + refresh flow	1.5	A1
A4	Extract role/department permission constants into single config	1	—
A5	Create environment config (`.env` files for dev/staging/prod)	0.5	—
A6	Remove `window.location.reload()`; replace with state updates	1.5	A1
A7	Remove all `console.log` statements	0.5	—
A8	Testing + regression check of foundation changes	1.5	A1-A7
	Phase A Total	**10 days**	

Phase B: Critical Modules (Week 3-5) — 15 working days

#	Task	Est. Days	Dependencies
B1	**10-access-management**: Fix 13-field payload, status polarity, roleMap, roleFilter null bug	3	A1-A4
B2	**Auth flow**: login, JWT handling, ProtectedRoute, role guards → backend permissions	2.5	A1-A4
B3	**03-department** + **13-primary-department**: User shape normalization, CRUD payloads	2	A1-A4

#	Task	Est. Days	Dependencies
E1	Full regression testing across all 23 pages	3	All
E2	Edge case testing: auth expiry, role changes, empty states, pagination	1.5	E1
E3	Mobile/responsive testing	1	E1
E4	Performance testing (list pages, large datasets)	1	E1
E5	Bug fixes from testing	1.5	E1-E4
	Phase E Total	**8 days**	

Summary

Phase	Description	Days	Weeks
A	Foundation (API client, config, cleanup)	10	1-2
B	Critical modules (auth, access mgmt, dept, dashboard)	15	3-5
C	Reporting & analytics (efficiency, goals, reports)	10	6-7
D	Feature modules (sales, daily logs, PMS, feedback, task mgmt)	11.5	8-9
E	Testing & hardening	8	10-11
Total		**54.5 days**	**~11 weeks**

Critical Path

```
A1 (API client) → A2 (URL replacement) → A3 (auth) → B2 (auth flow) →
B1 (access mgmt) → B3 (dept CRUD) → B5 (entry approval) →
C2 (goals cutoff fix) → C5 (query report) → D4 (PMS) → E1 (regression)
```

Milestones

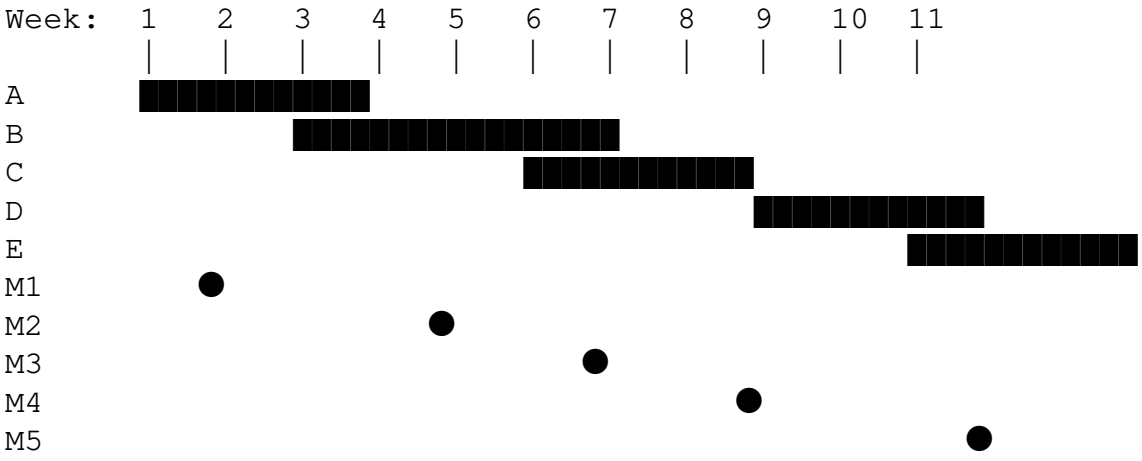
Milestone	Date (Week)	Criteria

M1: Foundation complete	Week 2	All pages use shared API client; no hardcoded URLs; auth refresh working
M2: Critical modules done	Week 5	User management, auth, departments, dashboard, entry approval working against Node.js
M3: Reporting done	Week 7	All efficiency/goal/report pages working against Node.js
M4: All modules done	Week 9	All 23 pages working against Node.js
M5: Production ready	Week 11	Full regression passed; zero critical defects

Risk Buffer

- +**3 days** buffer (total 56 days) for unforeseen backend API differences
- If backend migration is delayed, frontend work on Phase C/D can proceed with mock data

Gantt Chart (simplified)



Open Items & Unclear Questions (PHASE 7)

UNCLEAR — Backend Migration Questions

#	Question	Impact	Affected Pages	A
U1	What happens to `/list-efficiency-summary` field names (`AvgTargetPercent`, etc.) in Node.js?	8+ pages show summary cards	01,04,05,07,08,09,14,15	B te
U2	Does Node.js backend treat `page=0&size=0` as "fetch all" or "fetch none"?	All list pages may return empty	14+ pages	B te
U3	Will role IDs remain the same (1 = Root, 2 = User, 3 = EL, 4 = FL, 5 = Intern, 6 = Executive, 7 = Manager, 23 = Lead)?	All role-based guards break	All 23 pages	B te
U4	Will department IDs remain the same (21 = Sales, 22 = HR, 24 = Tech, 16 = Report)?	Department guards break	10,11,12,14,17,18	B te
U5	What is the polarity of `updateUserStatus` — is 1 = Active or 1 = Inactive?	Status toggle may set wrong value	10	B te
U6	Will `/users/read` return single shape (`{departmentID}`) or dual shape (`{departments[]}`)?	Normalization code breaks or dead	6,10,13,14	B te
U7	Is `2026-06-30` cutoff still needed? Does backend consolidate the dual API paths (efficiency summary vs userGoalData)?	Goals/reports show wrong data after cutoff	04,08,14,15	P o
U8	Does Node.js backend use same timezone as Java backend?	Date comparisons, zoned date pickers	01,04,08,09,11,14,15,16,17	B te
U9	Will status string values stay the same (ACTIVE, PENDING, RESOLVED, SCHEDULED, etc.)?	Status filters show wrong data	11,17,18	B te

U10	Will the login response keep same structure (`token`, `user` with `userRole`, `departmentID`, `departments[]`)?	Entire auth flow breaks	All	B te
U11	What format will permissions use in Node.js? Currently read from `localStorage.roles.rolePrivileges` (JSON)?	PMS tab visibility breaks	12	B te
U12	Will `/save-targets` keep `targets[{departmentID, userID, targetPercent}]` array format?	Target saving fails	16	B te
U13	Will user create/update keep all 13 fields (`employeeID`, `userName`, `userEmail`, `password`, `departmentID`, `role`, `roleID`, `secondaryRoleID`, `reportingManagerId`, `elManagerId`, `appraisalCycle`, `feedbackresolverteam`)?	User management fails	10	B te
U14	Advanced filter format in `/list-efficiency-summary`: `{field, operator, value}` — field names must match exactly?	Query report filters fail	14	B te
U15	What will the new backend API base URL be?	All API calls fail	All	D B

Decisions Needed

#	Decision	Options	Recommendation
D1	Move efficiency level calculation to backend?	(a) Keep in frontend, (b) Move to backend, (c) Backend returns raw fields + frontend computes	(b) Move to backend — single source of truth
D2	Replace localStorage permissions with API-fetched?	(a) Keep localStorage, (b) Fetch on login, (c) Fetch on each page load	(b) Fetch on login + refresh on token renewal

D3	Introduce TypeScript enums for roles/departments/status?	(a) Yes, (b) Keep string/number literals	(a) Yes — prevents silent failures
D4	Add i18n support during migration?	(a) Yes, (b) No, defer	(b) Defer — migration is complex enough
D5	Use React Query/SWR for data fetching?	(a) Yes, (b) Keep useEffect pattern	(b) Defer — too much refactoring for migration
D6	Add `window.location.reload()` removal — replace with what?	(a) State update + re-fetch, (b) React Query invalidation	(a) State update — minimal change
D7	Should frontend keep dual user shape normalization or standardize?	(a) Keep both, (b) Standardize to one after backend confirms	(b) Wait for backend U6 answer

Final Checklist Per Module

Module	API Payloads Aligned	Role/Dept Guards Updated	Status Constants Checked	Date Logic Verified	Console.log Removed	Tests
01-dashboard	☐	☐	☐	☐ (cutoff)	☐	☐
02-activity-log	☐	☐	☐	☐	☐	☐
03-department	☐	☐	☐	—	☐	☐
04-efficiency	☐	☐	☐	☐ (cutoff)	☐	☐
05-entry-approval	☐	☐	☐	☐	☐	☐
06-forms	☐	☐	☐	☐	☐	☐
07-goal-management	☐	☐	☐	☐	☐	☐
08-goals	☐	☐	☐	☐ (cutoff)	☐	☐
09-history	☐	☐	☐	☐	☐	☐
10-access-management	☐	☐	☐	—	☐	☐

11-sales-forward	☐	☐	☐	☐ (timezone)	☐	☐
12-pms	☐ (localStorage)	☐	☐	—	☐	☐
13-primary-department	☐	☐	☐	—	☐	☐
14-query-report	☐	☐	☐	☐ (cutoff)	☐	☐
15-reports	☐	☐	☐	☐ (cutoff)	☐	☐
16-target-management	☐	☐	☐	☐	☐	☐
17-daily-logs	☐	☐	☐	☐	☐	☐
18-hr-feedback	☐	☐	☐	☐	☐	☐
19-task-management	☐	☐	☐	☐	☐	☐

Next Steps

1. Share U1-U15 questions with backend team
2. Get decisions D1-D7 from tech lead/product owner
3. Set up staging environment with Node.js backend
4. Begin Phase A (Foundation) from timeline